

RL²-VLA: Adaptive RL Latent Compositional Steering with Test-Time Scaling for Vision-Language-Action Models

Derek Ming Siang Tan^{1,3†} Shailesh Shailesh^{1†} Srikrishna Iyer³ William Wei Jie Teo^{1,3}
Yuanliang Ju² Qiao Gu² Guillaume Sartoretti¹

¹National University of Singapore ²University of Toronto ³Singapore Technologies Engineering

<https://rl2-vla.github.io>

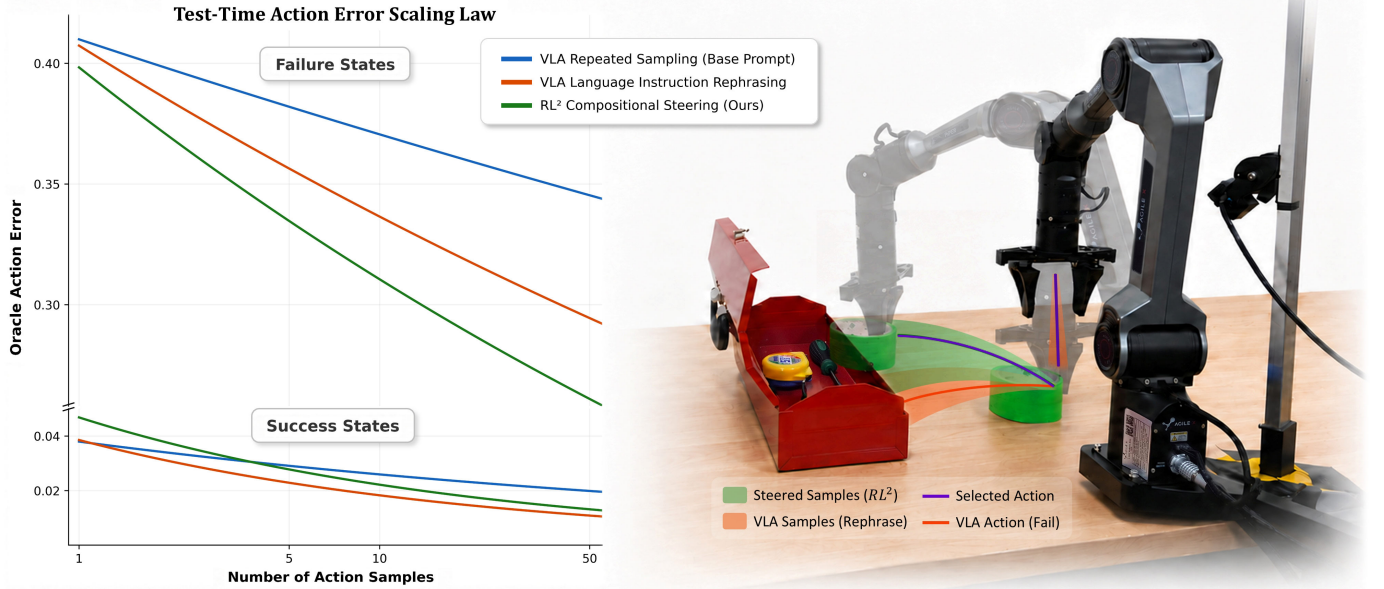


Fig. 1. **Overview:** RL^2 improves VLA test-time scaling by adaptively applying RL compositional steering when the base VLA is likely to fail, particularly in OOD settings, without modifying the pretrained VLA. **(Left)** Our scaling laws indicate that RL compositional steering (green) reduces action error most effectively under failure states, but can unnecessarily perturb already-accurate success states, motivating adaptive steering. **(Right)** During deployment, RL^2 initially follows the base VLA (orange) to accurately grasp the tape, then performs compositional steering (green) after impending failure is detected, generating diverse action candidates that lift the tape sufficiently high for successful placement into the toolbox (lighting and viewpoint slightly enhanced via GenAI).

Abstract—Despite the impressive visuomotor capabilities enabled by Vision-Language-Action (VLA) models, their performance often degrades on challenging and out-of-domain tasks. Recent test-time steering and scaling methods improve performance without extensive data collection and retraining, but action samples often remain concentrated around similar behaviors and therefore inherit correlated failure modes. Moreover, existing methods apply the same intervention strategy at every timestep, regardless of whether the base policy is already likely to succeed. To address these limitations, we introduce RL^2 , an adaptive inference-time steering framework that leverages Reinforcement Learning on VLA Latents. First, we train a lightweight offline RL policy conditioned on expressive latents extracted from the VLA action expert and compose its flow velocity with that of the frozen VLA during inference. This compositional steering strategy combines the behavioral priors of large-scale imitation learning with the action diversity induced by offline RL beyond dominant demonstration modes. We further discover that inference-time steering follows fundamentally different scaling laws under success and failure states, revealing that action diversity is most beneficial when the base VLA is likely to fail, but can unnecessarily perturb already-accurate actions when success is likely. Building on this insight, RL^2 activates compositional steering only when failure is predicted. Across the SIMPLER and PolaRiS benchmarks, RL^2 improves success rates by up to +17.3% in out-of-domain settings, while ablations and scaling studies demonstrate the importance of latent representations and

RL training. Finally, real-world experiments demonstrate that these gains transfer beyond simulation, establishing RL^2 as a practical and modular steering framework for VLA deployment.

I. INTRODUCTION

GENERALIST policies have emerged as a promising paradigm for general-purpose robotics, with Vision-Language-Action models (VLAs) among the most prominent examples. By leveraging large-scale robot demonstrations and strong priors from Vision-Language Models (VLMs), VLAs have demonstrated increasingly robust and generalizable performance across a diverse range of manipulation tasks [3]–[6]. Significant effort has been devoted to improving these models during pretraining through data scaling [7] and architectural design [4], and during post-training through preference alignment [8] and chain-of-thought reasoning [9], [10]. More recently, attention has shifted toward improving VLAs during deployment. These approaches leverage additional computation at test time without requiring extensive data collection and retraining. They aim to improve robustness especially in out-of-domain (OOD) settings where VLA success rates often deteriorate [1], [2], as seen in Fig. 2 (e.g. 70.2% $\rightarrow$ 36.0%). Existing methods broadly follow two parallel research directions: inference-time steering and test-time scaling.

[†] denotes equal contribution in no particular order

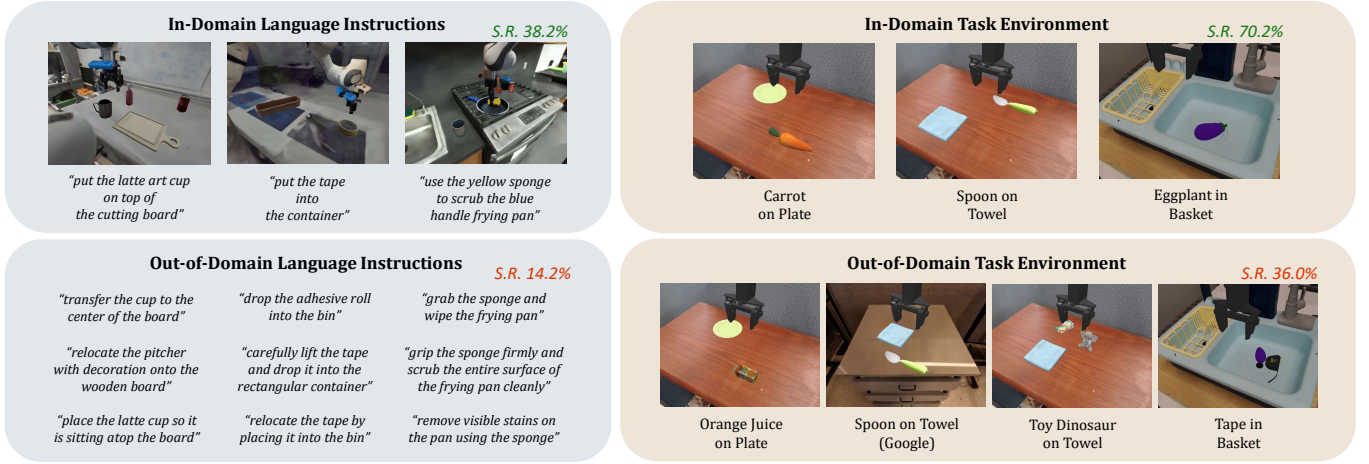


Fig. 2. **Types of In-domain and OOD Tasks:** While VLAs often perform well for in-domain tasks seen during training, their success rates tend to deteriorate when encountering unseen language instructions (left, 38.2% $\rightarrow$ 14.2%) [1] or unseen task environments (right, 70.2% $\rightarrow$ 36.0%) [2]. Recent methods focus on inference-time steering for VLAs to achieve better generalization to these OOD tasks without extensive data collection and expensive retraining.

Inference-time steering improves pretrained VLAs at deployment and can be broadly categorized into two directions. The first performs *discrete action selection* [11]–[15], sampling multiple candidate actions and selecting among them using external verifiers. While preserving the policy’s learned action distribution, these methods are limited by sample diversity, restricting adaptation to OOD scenarios. The second employs *differentiable steering* [16]–[18] to guide action generation through VLM-derived scoring functions or differentiable metrics computed from predicted future outcomes via world models. Although capable of steering policies beyond their original action distribution, these approaches are limited by imperfect physical grounding in pretrained VLMs and the difficulty of learning accurate world models [17]. In parallel, recent works have explored **test-time scaling** for VLAs to enhance inference-time steering. Early approaches repeatedly sample actions from the same VLA policy and rely on external verifiers to select the best candidate [11], [12], while more recent methods generate diverse action samples from rephrased language instructions [13]. Despite these advances, all generated samples ultimately originate from the same policy and may therefore inherit correlated biases and failure modes. More fundamentally, existing steering and scaling approaches apply a fixed intervention strategy and do not distinguish between situations where the base VLA is already likely to succeed and those where intervention may be necessary.

Humans, however, exhibit markedly different behavior: **When faced with uncertainty, humans actively broaden the diversity of alternatives they consider.** For example, when reaching for a mug on an uncluttered table, we typically grasp it directly without deliberating on alternatives. However, if the mug is partially occluded, we may approach it from another side or first remove the obstacle in the way. Thus, diverse alternatives are most useful when the default behavior is likely to fail, but may introduce unnecessary perturbations when the base solution is likely to succeed. This leads to the question:

How and when should we steer pretrained VLAs toward diverse action candidates to enhance robustness in challenging / OOD scenarios?

We address this with RL^2 , our proposed test-time steering

framework that leverages Reinforcement Learning on VLA Latents for adaptive compositional steering. To answer the question of **how to steer**, we introduce a lightweight RL-based flow-matching policy conditioned on expressive latents extracted from the VLA action expert. It is trained on the same offline finetuning dataset used by the VLA and introduces minimal computational overhead. Thereafter, we use the flow velocity of this RL policy to steer the flow velocity of the VLA action expert via composition (i.e., weighted average of velocities) [19]. We term this process **compositional steering**, which is particularly effective because it combines the benefits of imitation learning (IL) and RL. Specifically, the VLA contributes strong behavioral priors learned from large-scale but inherently imperfect demonstrations, while offline RL encourages the discovery of alternative high-value behaviors and induces greater action diversity beyond dominant demonstration modes (action sample illustrations in Fig. 11).

To answer the question of **when to steer**, we extend the test-time scaling laws established by existing works [12], [13] to steering approaches under success and failure states. Assuming an oracle verifier, we find that the relationship between normalized action error and the number of generated action samples follows an exponential power law. Our scaling law analysis reveals a key insight that aligns well with human intuition: **the diversity induced by compositional steering is most useful when the base VLA is likely to fail.** More specifically, compositional steering can avoid dominant failure modes, but may unnecessarily degrade already-accurate actions when success is likely. Motivated by this observation, we incorporate SAFE [20], a failure detector for VLAs, into RL^2 to adaptively enable compositional steering during failure and to fall back on the base VLA during success (Fig. 3). To the best of our knowledge, we are the first work to establish distinct test-time scaling laws that compare different steering approaches separately for success and failure states.

We evaluate RL^2 on both the SIMPLER [21] and PoLaRiS [22] benchmarks, covering diverse manipulation tasks under in-domain and OOD settings using different verifiers. Across these benchmarks, RL^2 consistently improves success rates by up to +17.3% over the strongest baselines for OOD

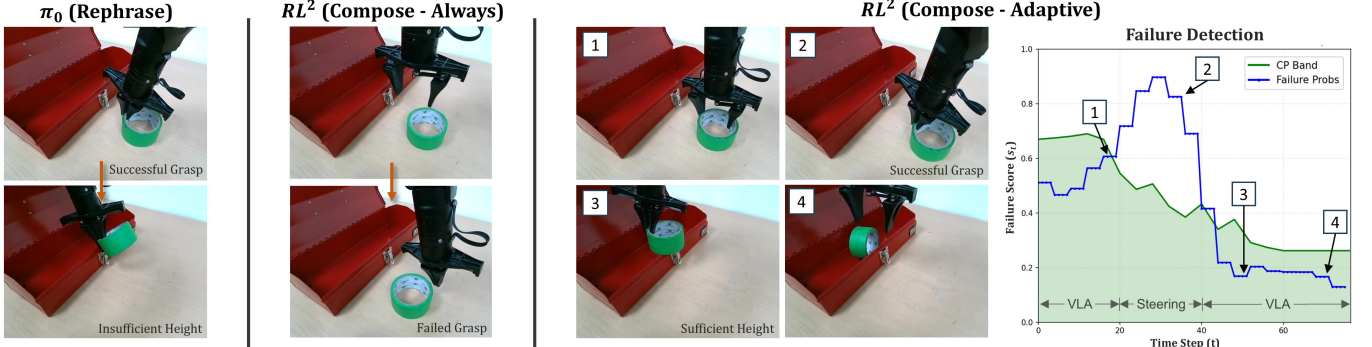


Fig. 3. **Real-Robot Experiment for Tape in Toolbox Task (OOD Environment):** This task is challenging for the base VLA (π_0 [4]) because the tape was never seen during training. **(Left)** Rephrase correctly grasps the tape but often collides with the side of the toolbox. **(Middle)** Non-adaptive RL^2 inaccurately approaches the tape, as it unnecessarily steers already-accurate VLA action samples. **(Right)** Adaptive RL^2 relies on accurate VLA samples to approach the tape, then steers actions to lift the tape high enough to avoid collision, when failure is preemptively detected via Conformal Prediction (CP) [20].

tasks. We further perform ablation studies to highlight the effectiveness of using VLA latents and RL training, as well as additional studies that demonstrate favorable scaling with both sample quantity and action diversity. Finally, we validate our approach on-hardware on a PiperX manipulator, where compositional steering improves success rate by an average of +17.5% over our strongest baseline, demonstrating the effectiveness and modularity of RL^2 for real-world deployment. Our main contributions are summarized as follows:

- We demonstrate that a lightweight offline RL policy conditioned on VLA latents and trained on the same VLA finetuning dataset can substantially improve downstream performance through compositional steering.
- We establish distinct test-time scaling laws for VLA steering approaches under success and failure states, revealing that RL compositional steering is beneficial during failure but degrades performance during successful states.
- We highlight the modularity of our lightweight RL^2 framework through extensive validation across diverse VLAs, verifiers, and manipulation benchmarks.

II. RELATED WORKS

A. Imitation vs Reinforcement Learning for VLAs

Many Vision-Language-Action (VLA) models are trained via behavior cloning on large-scale offline datasets [3]–[6]. While this paradigm has endowed policies with impressive generalization, they remain constrained by behaviors in the demonstration data. RL overcomes this limitation by enabling policies to discover high-value behaviors beyond demonstrations. One line of work uses RL to train entire VLA models end-to-end, such as RECAP ($\pi_{0.6}^*$) [23] and SimpleVLA-RL [24]. Although effective, updating large foundation models remains computationally expensive and often requires careful optimization to maintain training stability.

B. Inference-Time Steering

Inference-time steering methods aim to improve pretrained VLAs at deployment without updating the base model.

Discrete Action Selection: These methods generate multiple candidate actions and rely on external verifiers to select the most promising one. V-GPS [11] employs a learned value function to rank policy samples, while RoboMonkey [12] and CoVer [13] use significantly larger VLM-based verifiers

trained through preference learning and contrastive learning, respectively. FOREWARN [14] and Do What You Say [15] further augment verification with world-model rollouts. While these approaches preserve the strong behavioral priors learned from large-scale demonstrations, their ability to adapt to desired behaviors outside of this distribution remains limited.

Differentiable Steering: Rather than selecting among a fixed set of action candidates, differentiable steering methods directly influence the action-generation process using continuous guidance signals. DynaGuide [16] leverages a learned world model to predict future latent outcomes and steer actions according to guidance objectives, while VLS [17] and VLA-Pilot [18] employ VLM-derived scoring functions to guide the denoising process. Although these methods can steer policies beyond their original action distribution, they depend heavily on external guidance models; pretrained VLMs may lack physical grounding, while world models can introduce prediction errors and substantial inference cost [17].

Direct Action Refinement: Another line of work improves VLAs through lightweight auxiliary policies. ConRFT [25] freezes the VLA encoder and adapts an action head using consistency-based RL training objectives. Policy Decorator [26] and PLD [27] each learn a residual policy that outputs corrective offsets, while DSRL [28] uses RL to steer noise inputs of diffusion policies to enable efficient online adaptation. Similarly, RLT [29] learns compact tokens used by RL policies for rapid online learning. While effective, these methods often require online interaction with real or closely-simulated environments, increasing deployment cost.

Our approach steers the VLA using latent RL composition while retaining verifier-based selection. This combines the robustness of discrete action selection with the ability of differentiable steering to explore beyond the policy distribution.

C. Test-Time Scaling

Test-time scaling (TTS) aims to improve performance by allocating additional computation during inference. The first line of work improves inference-time steering methods by applying TTS to action generation. RoboMonkey [12] characterizes the first inference-time scaling law for VLAs through repeated action sampling and verifier-based selection, while CoVer [13] promotes additional diversity by sampling actions conditioned on multiple language instruction rephrases. However, additional samples drawn from the same VLA policy may simply

generate diverse variants of the same failure mode. Another line of work uses embodied chain-of-thought reasoning [9], [10] during inference to decompose long-horizon tasks into intermediate plans before generating robot actions. Our approach extends TTS for action generation by characterizing distinct scaling behaviors for steering approaches under success and failure states, and introduces latent RL steering to diversify candidate actions only during failure when it helps most.

D. Adaptive Inference for VLAs

Recent works on adaptive inference for VLAs aim to dynamically adjust the inference process according to the difficulty of the current state. Existing approaches primarily differ in the signals used to trigger adaptation. OneTwoVLA [30] learns routing tokens that determine whether the model should reason or act directly, while Recurrent-Depth VLA [31] dynamically adjusts latent refinement depth according to action consistency. Under high uncertainty, VLA-ATTC [32] invokes a deliberative action-selection process, whereas SCALE [33] broadens visual attention and action diversity. In contrast, we adopt a failure-centric perspective using SAFE [20], a state-of-the-art failure detector which outperforms uncertainty-based methods. Similar to SCALE, our goal is to diversify behaviors when failures are anticipated; however, rather than broadening exploration by adjusting the temperature hyperparameter of the same base policy, we employ a separately-trained lightweight RL policy to compositionally steer VLA actions.

III. PRELIMINARIES

A. Imitation and Reinforcement Learning

VLA models are typically trained via imitation learning on large-scale robot demonstration datasets $\mathcal{D} = \{o_i, a_i, \ell_i\}_{i=1}^N$, where o_i denotes the robot observation, a_i the corresponding action, and ℓ_i the task language instruction. Depending on the VLA architecture, observations may consist of one or more images together with robot proprioception, while the policy typically predicts an action chunk $a_{t:t+H}$ over a horizon H . The policy π_θ is trained by maximizing the log-likelihood of action chunks, $\max_\theta \mathbb{E}_{(a_{t:t+H}, o_t, \ell_t) \sim \mathcal{D}} [\log(\pi_\theta(a_{t:t+H} | o_t, \ell_t))]$. This objective encourages the policy to imitate the action distribution present in large-scale offline datasets.

In addition, we consider the standard RL formulation, modeled as a partially observable Markov Decision Process (POMDP). At each decision step, the agent selects and takes an action based on its policy $\pi_\theta(a_{t:t+H} | o_t, \ell_t)$. The overall goal of RL is to maximize the expected cumulative return $R = \mathbb{E}_{a_{t:t+H} \sim \pi(\cdot | o_t, \ell_t)} \left[\sum_{t=1}^T \gamma^{t-1} r^t \right]$.

B. Classifier Guidance for Flow-Matching Policies

Classifier guidance [34] provides a mechanism for steering pretrained generalist policies during inference. For a VLA with a flow matching head, action generation is governed by a velocity field $v(o, a, \ell, k)$ that progressively transports an initial noise sample toward a valid action trajectory. The flow-matching process models an Ordinary Differential Equation (ODE), where k denotes the flow-matching progress. Given a

differentiable guidance function $g(o, a, \ell)$ encoding a desired steering objective, the velocity field can be modified as:

$$\hat{v} = v(o_t, a_{t:t+T}, \ell_t, k) + \lambda \cdot g(o_t, a_{t:t+T}, \ell_t). \quad (1)$$

where λ controls the strength of guidance. Intuitively, the guidance function biases the flow matching process toward action trajectories that better satisfy a desired objective while preserving the underlying policy prior.

IV. INFERENCE-TIME SCALING LAW

We extend the test-time scaling laws established by existing works [12], [13] to various steering approaches under success and failure states, with the goal of characterizing how generated sample count affects action error across methods and states. Using π_0 [4] as our base VLA, we first generate the policy actions for each tuple (s, a^*, I) within the validation set of the large-scale BridgeV2 dataset [36]. We then compute the normalized root-mean-squared error (NRMSE) between policy and ground truth actions, and extract the top and bottom 1,024 tuples based on NRMSE to define the failure and success tuple sets respectively. The NRMSE formula per tuple is as follows:

$$\text{NRMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\frac{a_i - a_i^*}{\max_T a_i - \min_T a_i^*} \right)^2}. \quad (2)$$

where a denotes the policy action, a^* the ground truth actions, N the number of flattened action dimensions, and T tuples used from BridgeV2 dataset. Note that while there exist failure detection methods that can be trained using only successful trajectories from offline datasets, our approach offers a simpler, non-training-based alternative to determine policy failure since we have access to ground truth actions in BridgeV2.

We evaluate the following baseline steering approaches by taking the average NRMSE across each of the tuple sets:

- 1) **Repeated** [12]: actions are repeatedly sampled from the VLA $\pi(a | o, l)$ using the same language instruction l .
- 2) **Rephrase** [13]: actions are sampled from the VLA $\pi(a | o, l)$ where input l are VLM-generated rephrases.
- 3) **RBF** [17]: actions are sampled similarly to *Rephrase*, but the first n flow matching steps use Radial Basis Function (RBF) repulsion [35] to promote diversity.

We further evaluate other offline RL steering approaches, conditioned on rephrases and latents e from the VLA action expert (more details regarding e in Appendix A.2).

- 4) **RL** [37]: actions are repeatedly sampled from an RL flow-matching policy $\pi(a | e)$ trained using *Q-learning with Adjoint Matching* (QAM).
- 5) **Concat**: actions are sampled equally from both the base VLA and the RL flow-matching policy.
- 6) **Residual** [26]: residual actions are sampled from an offline RL policy $\pi_{res}(a | e, a_{VLA})$, and are then added to steer the base VLA actions. We adopt the implementation from [26], but trained using offline RL.
- 7) **RLT** [29]: actions are sampled from an offline RL policy $\pi(a | e, a_{VLA})$, also conditioned on VLA actions.
- 8) **RL²** (Ours): actions are generated by applying compositional steering to the base VLA samples (Sec. V).

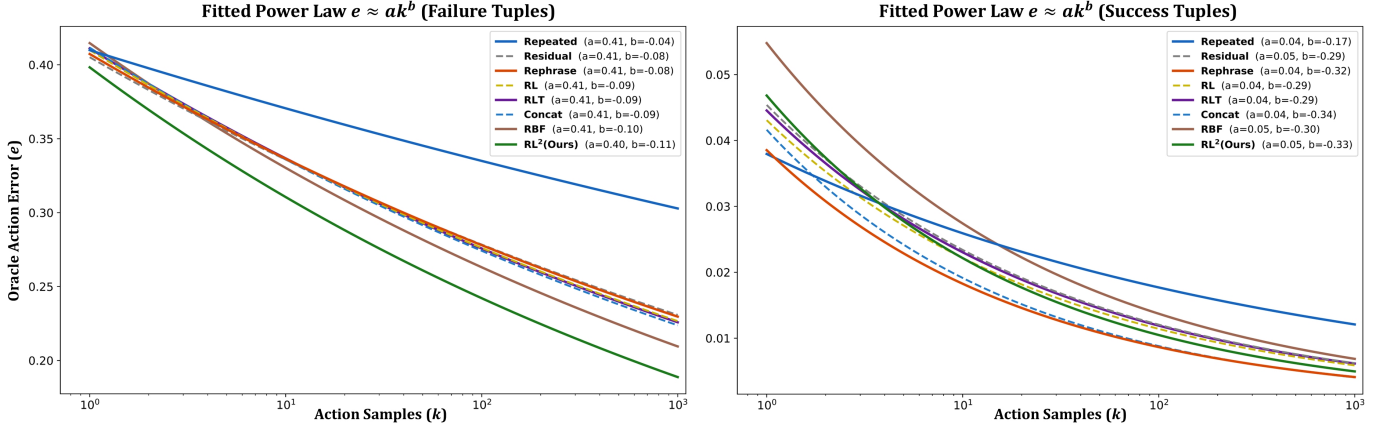


Fig. 4. **Test-time Scaling Laws (Steering Improves Failure States but Degrades Success States):** We observe that the action error between ground truth and predicted VLA actions (π_0 [4]) consistently decreases as we scale the number of action samples across different offline RL and baseline steering approaches. Most notably, compositional steering (RL^2) and RBF [35] exhibit the best scaling property due to their ability to generate diverse yet high-quality action samples during failure. However, they perform poorly compared to baseline approaches during success states, prompting the need to apply steering adaptively.

Similar to existing test-time scaling studies [12], [13], we find that the relationship between the normalized action error and the number of generated action samples follows an exponential power law, assuming an oracle verifier. For failure tuples, RL^2 compositional steering exhibits the strongest scaling behavior with significantly lower action error, indicating the importance of diversity during failure. Even simple diversity-inducing methods like RBF perform competitively, but fall behind RL^2 steering, likely because diversity is generated from the same base VLA. Most of the other methods outperform the *Rephrase* baseline only by a tiny margin.

In contrast, under success tuples, most methods exhibit the opposite trend. They consistently perform worse than the *Rephrase* baseline, with RBF and RL^2 among the worst. These results indicate that diversity-inducing methods like compositional steering are most useful when the base VLA is likely to fail; RL-induced diversity can offer failure-avoidant or recovery actions beyond dominant VLA failure modes, but can unnecessarily perturb already-accurate actions when success is likely. To the best of our knowledge, we are the first work to establish distinct test-time scaling laws comparing steering methods separately for success and failure tuples.

V. METHOD

We introduce RL^2 , a framework to answer the question of **how** and **when** we should steer our pretrained VLAs towards diverse action candidates. More specifically, we enhance our action samples with RL compositional steering, only when our VLA is deemed by an external failure detection module to be failing. We present the overall framework and pseudo-code of RL^2 in Fig. 5 and Algorithm 1 respectively.

A. RL Latent Policy Training

To generate a differentiable guidance function $g(o, a, \ell)$ for steering VLA action samples, we first train a steering RL flow-matching policy $\pi_{RL}(a_{t:t+H} | e_t)$ conditioned on latents extracted from the VLA action expert like in [20]. We describe how these latents are aggregated into a single embedding vector e in Sec. V-C. To construct training data for RL policies

conditioned on e , we augment existing large-scale datasets, namely BridgeV2 [36] and DROID [38], with policy latents to obtain $\mathcal{D} = \{o_i, a_i, \ell_i, e_i\}_{i=1}^N$. Specifically, we iterate through each dataset tuple, pass its observations through the VLA, and extract the resulting embeddings.

Training flow-matching actor-critic networks is challenging because direct backpropagation from the critic $Q(s, a)$ through the policy’s multi-step flow-matching process is numerically unstable [39]. To mitigate this, we leverage QAM [37], which replaces unstable backpropagation with a stable, step-wise matching objective. QAM utilizes a lean adjoint state $\tilde{g}_t$ to act as a time-dependent guidance signal. This state is computed backwards from the terminal action a_1 via a reverse ODE using a fixed behavior prior f_β , an inverse temperature τ , and with terminal condition $\tilde{g}_1 = -\tau \nabla_{a_1} Q(s, a_1)$:

$$d\tilde{g}_t = -\nabla_{a_t} [2f_\beta(s, a_t, t) - a_t/t] \tilde{g}_t dt \quad (3)$$

The policy’s target velocity field f_θ is then optimized to align with this signal by minimizing the matching loss $L_{AM}(\theta) =$

$$\mathbb{E}_{s, \{a_t\}} \int_0^1 \left\| \frac{2(f_\theta(s, a_t, t) - f_\beta(s, a_t, t))}{\sigma_t} + \sigma_t \tilde{g}_t \right\|_2^2 dt, \quad (4)$$

where $\sigma_t = \sqrt{2(1-t)/t}$ is a fixed noise schedule. This formulation ensures that the policy converges to the optimal behavior-regularized distribution $\pi(a|s) \propto \pi_\beta(a|s) \exp(\tau Q(s, a))$ while maintaining full expressivity through a stable, first-order optimization path. In practice, and consistent with the findings in [37], we find that simply adjusting the inverse temperature τ is sufficient to achieve superior performance over behavior cloning (BC) baselines (see Sec. VI-C2 for ablations).

Note that our RL^2 framework is compatible with different RL methods that are composable with the VLA. This includes non-flow-based methods like the policy trained in V-GPS [11] via Conservative Q-Learning (CQL) [40] that are compatible with autoregressive VLAs such as OpenVLA [3]. We train our RL policies on two NVIDIA L40s GPUs for 500k-1M steps, with training times largely determined by the VLA latents dimensionality (details in Appendix A.1).

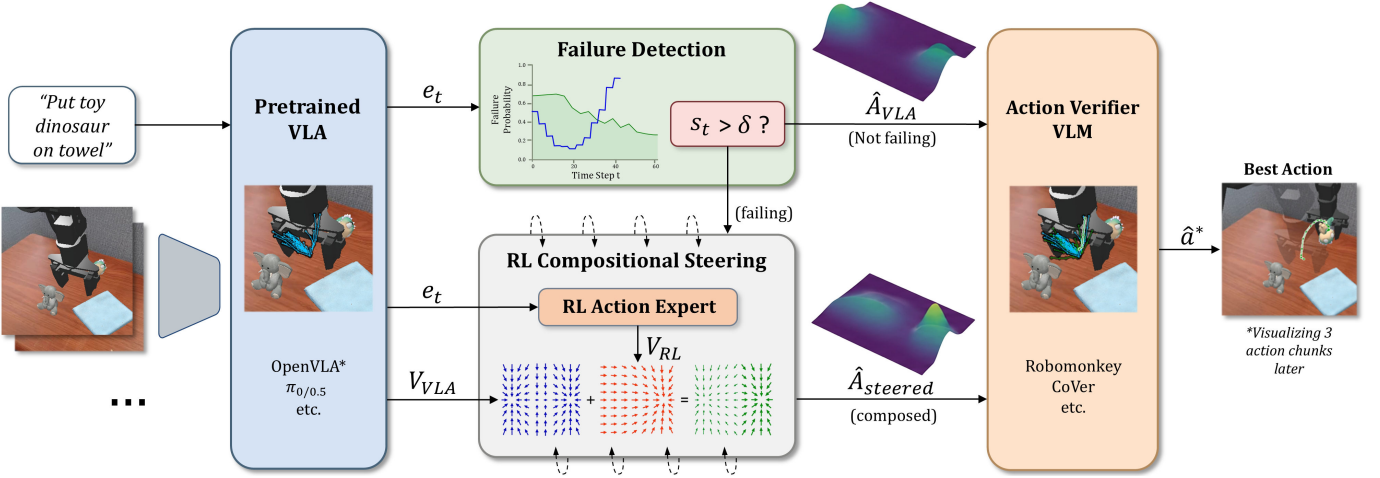


Fig. 5. **RL² Framework (Flow-Matching VLAs):** (i) For flow-matching-based VLAs like π_0 [4], given language instructions and observations, RL² first extracts action expert latents (e_t) from its frozen VLA action expert. (ii) The latents e_t is fed into the RL flow-matching steering policy, which provides guidance velocities V_{RL} that are composed with the VLA flow velocities V_{VLA} at each flow-matching step to generate diverse and high-quality action candidates. (iii) In parallel, e_t is passed into a failure detection module that determines whether steering should be enabled: if failure is detected, RL² composes the VLA and RL velocities; otherwise, it falls back to the base VLA action distribution. (iv) Finally, a verifier scores the generated candidates and selects the best action to execute (*for autoregressive VLAs like OpenVLA [3], we use Gaussian perturbation [12] to perform RL compositional steering).

B. Compositional Steering

In order to steer the frozen VLA action distribution $\pi_{VLA}(a_{t:t+H} | o_t, \ell_t)$ toward candidates that are more diverse and of higher quality, we use the RL flow-matching head as an auxiliary steering function $\pi_{RL}(a_{t:t+H} | e_t)$. Following [19], we inject this guidance during each step of the VLA flow-matching process by composing the VLA and RL policies as a weighted average of their velocity fields. Specifically, if v_{VLA} denotes the VLA flow-matching velocity and v_{RL} denotes the RL-induced steering velocity, the enhanced VLA velocity is $v_{comp} \leftarrow wv_{VLA} + (1-w)v_{RL}$. We draw composition weights w from a Gaussian distribution ($\mu = 0.5$, $\sigma = 0.25$), such that action samples can benefit from both the demonstration-aligned action priors from VLAs and the diversity from RL steering by varying extents.

For autoregressive VLAs such as OpenVLA [3], they cannot be steered similarly using QAM because both action heads must be flow-matching policies. We instead adopt the Gaussian perturbation strategy from [12]; we first fit a Gaussian distribution over an equal mix of VLA and RL action samples, and then sample a larger action batch from this fitted distribution.

C. Failure Detection for Adaptive Steering

Multi-Task Failure Detector: As established in Sec. IV, compositional steering is most effective only when the base VLA is likely to fail, but can be unnecessary or even detrimental when the base policy already produces feasible actions. We therefore introduce a failure-detection trigger that enables steering only at timesteps when the VLA is detected to be failing, while otherwise falling back to the original VLA action distribution. We instantiate this trigger with SAFE [20], a lightweight multitask failure detector conditioned on informative internal VLA features. For each task, we collect 100 rollouts per seed across three random seeds, yielding trajectories that contain both successes and failures, and split them into training and validation sets $\mathcal{D}_{train}$ and $\mathcal{D}_{val}$ using a 60%/40% split. Following SAFE, we adopt an LSTM

Algorithm 1 RL²: Adaptive RL Compositional Steering

Notation: Uppercase symbols represent batches of size N (e.g., $\hat{A}_t = \{\hat{a}_i^t\}_{i=1}^N$).

Require: Base VLA π_{VLA} , RL steering policy π_{RL} , failure detector f_ψ , CP threshold δ_t , verifier $\mathcal{V}_\theta$, instruction l_t , observation o_t , batch size N

- 1: Extract VLA feature embedding e_t // from action expert
- 2: Predict LSTM failure score $s_t \leftarrow f_\psi(e_{0:t})$
- 3: **if** $s_t > \delta_t$ // failure
- 4: Sample compose weights $W \sim \mathcal{N}(0.5, 0.25)$ // clipped [0,1]
- 5: Initialize noisy actions $\mathcal{T}_T \sim \mathcal{N}(0, I)$
- 6: **for** $k = T, \dots, 1$ // flow matching steps
- 7: $V_{VLA} \leftarrow \pi_{VLA}(\mathcal{T}_k, k, o_t, l_t)$
- 8: $V_{RL} \leftarrow \pi_{RL}(\mathcal{T}_k, k, e_t)$
- 9: $V_{comp} \leftarrow WV_{VLA} + (1-W)V_{RL}$ // velocity composition
- 10: $\mathcal{T}_{k-1} \leftarrow \text{Step}(\mathcal{T}_k, V_{comp}, k)$ // flow step
- 11: $\hat{A}_t \leftarrow \mathcal{T}_0$ // steered action samples
- 12: **else**
- 13: Sample VLA candidates $\hat{A}_t \sim \pi_{VLA}(\cdot | o_t, l_t)$
- 14: Score candidates $R_t \leftarrow \mathcal{V}_\theta(o_t, \hat{A}_t, l_t)$
- 15: Select $\hat{a}_t^* \leftarrow \hat{a}_t^{n^*}$, where $n^* = \arg \max_n R_t^n$ // best action
- 16: Execute $\hat{a}_t^*$

detector f_ψ that takes in VLA features up to the current timestep and outputs a failure score $s_t = f_\psi(e_{0:t}) \in [0, 1]$. The per-timestep feature e_t is obtained by extracting hidden-state vectors from the VLA action expert; we ablate different aggregation strategies (Appendix A.2) for converting these internal features into a single embedding vector e_t and select the best one according to $\mathcal{D}_{val}$ performance. We train f_ψ using a binary cross-entropy loss applied across all timesteps, $\mathcal{L}_{BCE} = \sum_i \sum_t [y_i \log s_t + (1 - y_i) \log(1 - s_t)]$.

Conformal Prediction (CP): Steering is triggered by a simple threshold rule: at timestep t , we intervene whenever the predicted failure score exceeds a time-varying threshold δ_t . We calibrate δ_t using conformal prediction (CP), adopting the

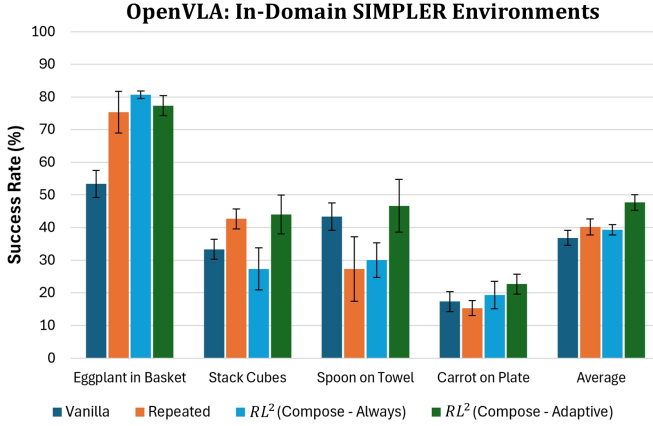


Fig. 6. **SIMPLER In-Domain Evaluation:** Using OpenVLA [3], adaptive RL^2 improves performance on standard SIMPLER tasks [21], by up to +19.4% in task-wise success rate (average +7.5%) over *Repeated* baseline.

one-sided time-varying CP formulation following [41]. Given a rollout-level score sequence $s_{1:T}$ and significance level $\alpha \in (0, 1)$, the CP band is defined as $C_\alpha = \{[lower_t, upper_t] : t = 1, \dots, T\}$, with $lower_t = -\infty$ and $upper_t = \mu_t + h_t$, where μ_t and h_t are the time-varying mean failure score and conformal bandwidth, respectively. We calibrate this band using successful rollouts from $\mathcal{D}_{val}$, so that any new successful rollout satisfies $s_t < \mu_t + h_t$ for all timesteps with probability at least $1 - \alpha$. Note that while we train a single failure detector on all tasks, we calibrate per-task CP bands and a combined CP band for in-domain and OOD tasks respectively. More details and illustrations can be found in Appendix C.1.

CP Alpha Selection Heuristic: In practice, we empirically observe that the best CP significance level α varies across tasks because α directly controls the frequency with which steering is triggered. Rather than exhaustively sweeping all possible α values for every task which would require costly evaluations (like in [19] for composition weights), we instead introduce a *test-time alpha selection heuristic*. On $\mathcal{D}_{val}$, we evaluate candidate α values by balanced accuracy, $BalAcc(\alpha) = \frac{1}{2}(TPR(\alpha) + TNR(\alpha))$, where TPR measures the fraction of failed rollouts correctly flagged as failures and TNR measures the fraction of successful rollouts correctly flagged as success. We then perform a calibration sweep for the top three α values under this metric and select the best-performing value for final evaluation. We find that our choice of balanced accuracy as an α selection metric correlates well with downstream task performance (Sec. VI-D2). More details and illustrations of our heuristic can be found in Appendix C.2.

D. Action Verification

Given a set of N candidate actions $\hat{A} = \{\hat{a}_t^1, \dots, \hat{a}_t^N\}$, where $\hat{a}$ refers to either VLA or steered action samples, an action verifier serves as a reward model to select the best action sample to execute for each timestep t . More specifically, the verifier assigns each candidate a score $r_t^n = \mathcal{V}_\theta(o_t, \hat{a}_t^n, l_t)$, and selects the highest scoring sample $\hat{a}_t^* = \arg \max_n(r_t^n)$. RL^2 is designed to be adaptable to different types of action verifiers. This ranges from verifiers trained via preference learning such as RoboMonkey [12] to verifiers trained via contrastive learning such as CoVer [13].

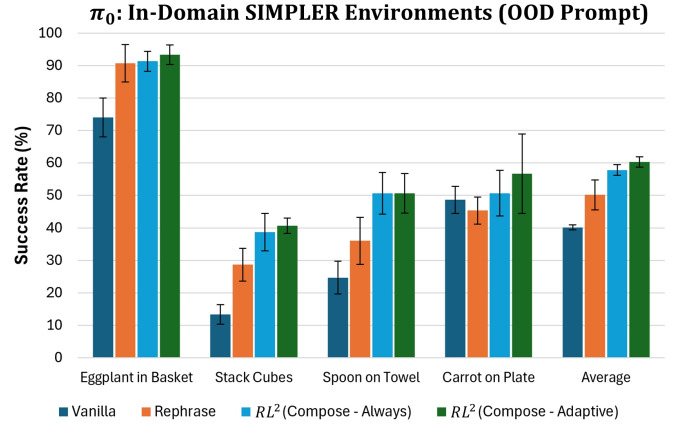


Fig. 7. **SIMPLER OOD Prompt Evaluation:** Using π_0 [4], adaptive RL^2 improves robustness under OOD language instructions [1], achieving up to +14.7% in task-wise success rate (average +10.1%) over *Rephrase* baseline.

VI. EXPERIMENTS

The primary objective of our experiments is to test the robustness and generalization of RL^2 to challenging or OOD tasks, while providing a modular interface compatible with various pretrained VLAs and verifiers. While this work tackles several research questions, the main ones are:

- Does compositional steering improve task performance?
- How does using latents and RL enable effective steering?
- How well does RL^2 scale with different numbers and types of action samples?

A. Experimental Setup

We evaluate RL^2 across a diverse set of simulation benchmarks, VLAs, and verifiers. In addition to testing the robustness of our framework on standard benchmarks, we also focus on tasks that assess RL^2 's generalization to OOD language instructions and task environments. To achieve this, we adopt the experimental framework from prior works on test-time scaling [12], [13]. These tasks are visualized in Fig. 2, and more details can be found in Appendix B.

SIMPLER (Original): The SIMPLER benchmark [21] is a simulation environment with in-domain manipulation tasks mimicking demonstration setups from large-scale datasets. In this environment, we test OpenVLA [3] on four of their standard in-domain tasks from BridgeV2 [36] (WidowX robot) with their original task prompts. We use the policy trained in V-GPS [11] via CQL [40] as our RL steering policy and RoboMonkey [12] as our verifier.

SIMPLER (OOD): We test π_0 using the SIMPLER benchmark with the same four in-domain tasks, but with challenging red-teaming instructions [42] to simulate OOD base language prompts, via the implementation in [1]. We also introduce four additional OOD task environments from Interleave-VLA [2], which offer a mix of varying backgrounds, task objects, and the presence of distractors. All of these OOD tasks assess if VLAs can enhance task performance using VLM-generated rephrases. Here, we use QAM [37] as our RL flow-based steering policy and CoVer [13] as our verifier.

Polaris (OOD): We test $\pi_{0.5}$ on the Polaris benchmark [22], which offers high-fidelity Gaussian-splat simulation

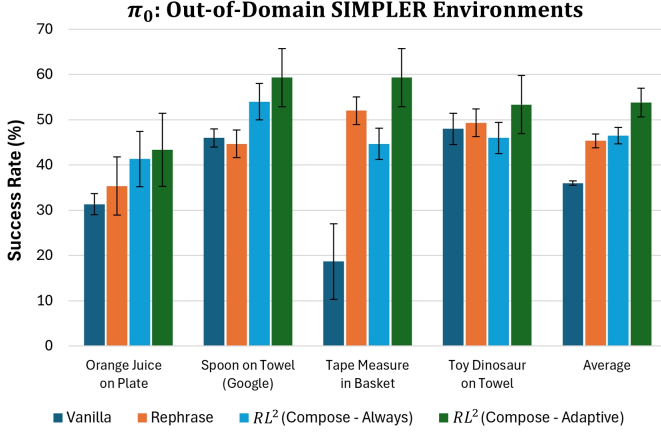


Fig. 8. **SIMPLER OOD Environment Evaluation:** Using π_0 [4], adaptive RL^2 improves robustness in OOD environments [1], by up to +14.6% in task-wise success rate (average +8.5%) over the strongest *Rephrase* baseline.

environments with a Franka Emika Panda robot. We evaluate our approach on three tasks adapted from the DROID dataset [38], but with OOD red-teaming base prompts. These tasks contain many other distractor objects in the scene, and assess if VLAs can enhance task performance using VLM-generated rephrases. Likewise, we use QAM [37] as our RL flow-based steering policy and CoVer [13] as our verifier.

Baselines: To evaluate RL^2 , we compare our approach with several baselines: (1) **Vanilla** represents either vanilla OpenVLA or $\pi_{0.5}$, (2) **Repeated/Rephrase** samples multiple actions from the VLA via repeated sampling [12] or VLM-generated rephrases [13], (3) RL^2 (**Compose-Always**) uses compositional steering on multiple actions sampled from the VLA, and (4) RL^2 (**Compose-Adaptive**) uses compositional steering only when the VLA is flagged as failing by SAFE [20] but uses *Repeated/Rephrase* otherwise. Note that for OpenVLA, we follow [12] which employs Gaussian perturbation by fitting a Gaussian distribution over 9 action samples (VLA and/or composed), then re-sampling 32 actions from it. In addition, for $\pi_{0.5}$, we follow [13] which uses 8 rephrases with 5 action samples each. We run all experiments 50 times each and report the mean and standard deviation across 3 random seeds. We will release our code with the final paper.

B. Simulation Results

We summarize our simulation results for both in-domain and OOD tasks, and report average and task-wise performance gains of up to +10.9% and +19.4% respectively over our strongest baseline. We believe these gains are significant given that we are only using offline RL. More baseline comparisons can be found in our NRMSE analysis (Sec. IV).

1) Compositional steering is robust to standard and OOD language prompts: To illustrate the robustness of RL^2 towards standard and OOD base prompts on tasks seen by the VLA, we present our simulation results for OpenVLA, π_0 , and $\pi_{0.5}$ in Figs. 6 and 7, and Table I. Based on the average performance across all benchmarks, we observe a general improvement in success rates with our adaptive RL^2 framework. For OpenVLA, adaptive RL^2 achieves an average performance gain of +7.5% (task-wise +19.4%) over the strongest *Repeated*

TABLE I
POLARIS OOD PROMPT EVALUATION: USING $\pi_{0.5}$ [5], ADAPTIVE RL^2 IMPROVES ROBUSTNESS UNDER OOD LANGUAGE INSTRUCTIONS [1], ACHIEVING UP TO +17.3% IN TASK-WISE SUCCESS RATE (S) AND +12.4% IN TASK-WISE PROGRESS RATE (P) OVER *Rephrase* BASELINE.

Task	Metric	$\pi_{0.5}$ (Vanilla)	$\pi_{0.5}$ (Rephrase)	RL^2 (Compose)	RL^2 (Adaptive)
Move Latte Cup	S (%)	18.7	48.7	55.3	66.0
	P (%)	39.5	65.6	72.0	78.0
Tape into Container	S (%)	12.7	22.0	22.0	28.7
	P (%)	34.6	41.3	40.9	46.9
Pan Cleaning	S (%)	11.5	24.7	24.0	33.3
	P (%)	45.7	58.7	54.4	64.0
Average	S (%)	14.3 (± 2.0)	31.8 (± 3.4)	33.8 (± 1.7)	42.7 (± 2.3)
	P (%)	40.0 (± 0.7)	55.2 (± 1.4)	55.8 (± 1.2)	63.0 (± 3.4)

baseline across SIMPLER in-domain tasks. For π_0 , adaptive RL^2 achieves an average performance gain of +10.1% over the strongest *Rephrase* baseline across SIMPLER tasks under OOD language prompts, with +14.7% gain for the highest performing task (*Spoon on Towel*). For $\pi_{0.5}$, adaptive RL^2 achieves an average performance gain of +10.9% and +7.8% on success and progress rates respectively, over the strongest *Rephrase* baseline across PolaRiS tasks under OOD language prompts (+17.3% success rate for *Move Latte Cup* task).

These results indicate the robustness of RL^2 to in-domain and OOD language prompts. We attribute this to the diversity introduced by RL compositional steering during failure states. As seen in Fig. 11, where we visualize action chunks from π_0 using rephrases for the *Spoon on Towel* task, the action samples tend to be uniformly directed away from the spoon. In contrast, the steered action samples tend to be more diverse and offer some higher-quality action samples towards the spoon, one of which is selected by the CoVer verifier. Across the episode, we observe that π_0 with rephrases leads to oscillations above the spoon, while RL^2 leads to accurate and decisive grasping of the spoon.

2) Compositional steering generalizes well to OOD task environments: To assess the generalization of RL^2 on task environments that were not seen by the VLA during training, we introduce four additional tasks within the SIMPLER environment with varying backgrounds, task objects, and the presence of distractors (Fig. 2 and Appendix B). We benchmark using π_0 across these SIMPLER tasks under OOD language prompts, and observe an average performance gain of +8.5% over the strongest *Rephrase* baseline, with up to +14.6% gain for the highest performing task (*Spoon on Towel - Google*). We can gain some insights into the strong performance of RL^2 in Fig. 11, where we can observe that *Rephrase* π_0 action samples tend to be directed mostly towards the wrong distractor object (i.e., toy elephant) during a failure state. In contrast, the RL-steered action samples offer diversity that skews the overall action distribution more towards the main task object (i.e., dinosaur), leading to accurate and decisive grasping.

C. Ablations

We perform a comprehensive series of ablation studies to verify that all components of RL^2 are crucial in improving downstream performance across various benchmarks.

TABLE II
ABLATION STUDIES: RL TRAINING AND VLA LATENTS ENHANCE COMPOSITIONAL STEERING (*IN-DOMAIN PROMPT FOR OPENVLA).

Model	Latents	Method	OOD Prompts (%)	OOD Tasks (%)
π_0 [4] (SIMPLER)	✓	RL	57.8 (± 1.6)	46.5 (± 1.8)
	✓	BC	55.8 (± 3.0)	42.0 (± 4.0)
$\pi_{0.5}$ [5] (PolaRiS)	✓	RL	33.8 (± 1.7)	—
	✓	BC	29.7 (± 4.7)	—
OpenVLA* [3] (SIMPLER)	✓	RL	39.3 (± 1.6)	—
	✗	RL	0.5 (± 0.5)	—

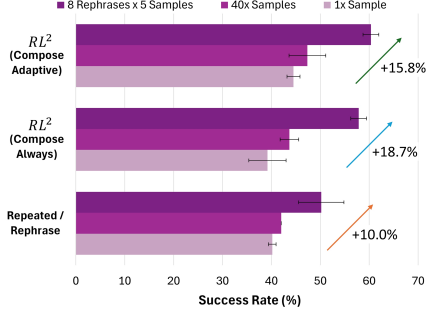


Fig. 9. **Scaling Samples and Rephrases:** RL^2 attains larger performance gain compared to baselines when scaling both samples and rephrases.

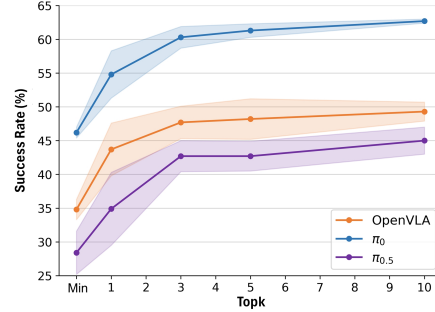


Fig. 10. **CP Alpha Selection Heuristic:** Our heuristic selects one of the top alphas within top-3 selection across all baselines.

1) **Adaptiveness is crucial for effective steering:** To assess the importance of adaptiveness in compositional steering, we perform an ablation study by removing the failure detection module in RL^2 (Figs. 6 and 7, Table I). While non-adaptive steering performs nearly on-par with adaptive steering for π_0 , the broader trend across OpenVLA and $\pi_{0.5}$ indicates that adaptive steering yields significantly stronger gains. More specifically, we observe an average and task-wise performance gain of up to +8.9% and +16.7% respectively. This aligns well with our NRMSE analysis (Sec. IV) where compositional steering is most beneficial only when applied to failure states.

2) **Using latents and RL results in effective steering:** We first perform an ablation by modifying the training recipe from RL to Behavior Cloning (BC). We measure the performance of composition at all times for $\pi_{0/0.5}$ in Table II, which indicates that using RL consistently outperforms BC by up to +4.5%. This supports our intuition that a different yet effective training recipe like RL can introduce the diversity required for better verifier action selection during VLA failure.

In addition, we perform an ablation for $\pi_{0/0.5}$ by conditioning our steering policy on raw observations instead of VLA latents (Table II). However, we were unable to successfully train QAM on raw observations in BridgeV2 as it was originally optimized for proprioceptive inputs on OGBench [43]. Hence, we test this with OpenVLA instead, using the CQL policy trained in V-GPS as its RL steering policy that is optimized for vision-language inputs. We observe that steering via latents achieves a performance gain of +38.8% over steering without latents. Notably, the vanilla V-GPS formulation conditioned on raw observations performed poorly at 0.5%. This large gap is likely due to the use of latents from the VLA action expert, which is significantly more expressive [20] than features from small encoders like ResNet used by vanilla V-GPS.

TABLE III
ABLATION STUDIES: SAFE [20] FAILURE DETECTOR IS AN EFFECTIVE AND TIMELY TRIGGER FOR COMPOSITIONAL STEERING.

Model	Trigger	OOD Prompts (%)	OOD Tasks (%)
π_0 [4] (SIMPLER)	SAFE [20]	60.3 (± 1.6)	53.8 (± 3.2)
	CoVer [13]	57.8 (± 1.3)	50.3 (± 1.2)
	Always	57.8 (± 1.6)	46.5 (± 1.8)
$\pi_{0.5}$ [5] (PolaRiS)	SAFE [20]	42.7 (± 2.3)	—
	CoVer [13]	39.8 (± 1.8)	—
	Always	33.8 (± 1.7)	—

TABLE IV
SCALING TIME ANALYSIS: RL STEERING AND FAILURE DETECTION REMAIN EFFICIENT WITH INCREASING BATCH SIZES (IN MS, ON NVIDIA RTX5090 GPU).

Batch Size	π_0 [4]	CoVer [13]	QAM [37]	SAFE [20]
1	232	96	12	1
2	243	100	15	1
4	265	102	16	1
8	320	107	17	2
16	434	111	21	2
32	724	114	27	2
64	1541	116	40	2
128	3093	145	48	2

3) **Failure detection methods outperform other triggers for adaptive compositional steering:** To assess the importance of SAFE as an effective trigger for compositional steering, we perform an ablation study by using an external VLM as the trigger instead. More specifically, we use the CoVer verifier [13] as our VLM for both failure detection and action selection. Similar to our RL^2 setup in Sec. V-C, we collect online rollouts with CoVer verifier scores in order to generate CP bands for failure detection. As seen from Table III, using CoVer as a failure detector generally leads to improvements over the baseline of steering at all times. Nevertheless, using SAFE still performs up to +3.5% better compared to using CoVer, likely because SAFE is conditioned on informative latents extracted directly from the action expert. Conversely, CoVer is a poorer failure detector, as seen from Appendix C.1 where success and failure scores are too correlated and thus not as separable as compared to using SAFE.

D. Additional Studies

We perform an additional series of experiments to validate the scalability and practicality of our RL^2 framework.

1) **RL^2 scales well with increasing quantity and diversity of samples:** From our NRMSE analysis in Sec. IV, we gain the insight that a VLA can achieve the best performance when we scale both the quantity and diversity of samples during failure states. We validate this insight by scaling the number of sampled actions and rephrases on the four SIMPLER tasks with OOD prompts using π_0 , with and without compositional steering. From Fig. 9, we notice two consistent trends that support our findings. Increasing the number of samples (1 $\rightarrow$ 40) for a given language prompt, and increasing the diversity of samples (1 $\rightarrow$ 8 rephrases) for the same total number of samples (40), both improve success rates across all methods.

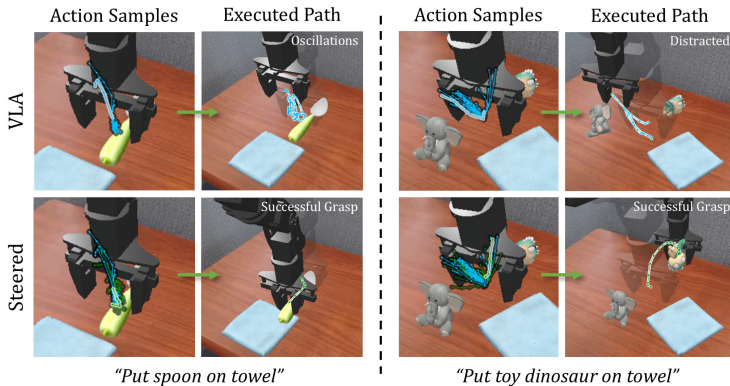


Fig. 11. **Action Samples Visualization** (π_0): During failure, VLA actions (blue) using rephrases tend to either be directed away from the task object or toward the distractor object, resulting in oscillatory behaviors. RL^2 performs compositional steering (green) to direct VLA samples toward the task object for successful grasping.

RL^2 outperforms *Repeated* and *Rephrase* by achieving up to +18.7% in performance gain when both types of scaling are combined, supporting our scaling laws established in Sec. IV.

2) **Our alpha selection heuristic is effective in proposing the top per-task alphas during test-time:** In Sec. V-C, we established the importance of designing an efficient heuristic that selects the optimal CP significance level α for each task, in order to introduce the appropriate amount of steering during failure. To validate the efficiency of our balanced-accuracy heuristic, we study the success rates for $\alpha = 0.05$ to $\alpha = 0.50$ in intervals of 0.05, and plot the performance gains as we pick the top-k α based on our heuristic, where we test $k = \{1, 3, 5, 10\}$. From Fig. 10, we observe that most of the performance gains, up to 88.5% relative to maximum performance, can be derived from simply picking the top-3 alphas using our heuristic. While there may exist a better metric beyond balanced accuracy (e.g., detection timeliness), we leave further improvements for future work.

3) **Our steering policy and failure detection module introduce minimal additional latency:** While RL^2 introduces new modules to the existing verifier-based action selection frameworks, these are lightweight and do not incur substantial latency compared to the baselines themselves. We illustrate this by measuring the inference time across the different components for the π_0 setup [13] in Table IV. As we scale the number of action samples, the inference time for our QAM steering policy and SAFE failure detector remains significantly lower compared to the VLA forward pass. This is mainly because these networks are lightweight and are conditioned on VLA latents, making them compatible with a wide range of verifier-based frameworks at minimal inference costs.

4) **Qualitative analysis highlights the efficacy of compositional steering:** To better understand the underlying reasons for the effectiveness of compositional steering, we visualize action chunks in our π_0 simulation (Fig. 11). As discussed in-depth in Sec. VI-B1 and VI-B2, the VLA samples tend to be directed away from the task object or towards other distractors when the VLA is failing, causing long-term oscillatory behaviors. Our RL^2 framework steers more samples towards the task object, providing diversity for verifier action selection during failure (more illustrations in Appendix Fig. 19–21).

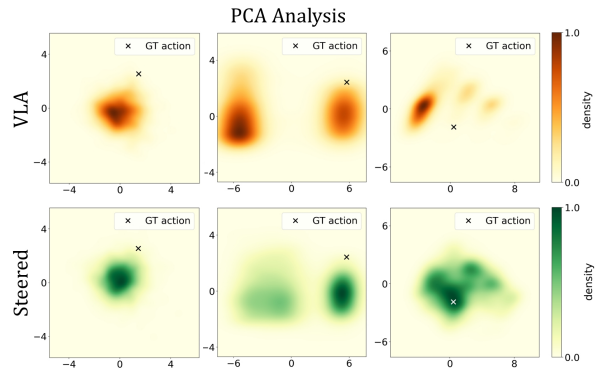


Fig. 12. **PCA Heatmap Analysis:** We represent VLA actions along its two principal axes via Principal Component Analysis on tuples from BridgeV2 dataset [36]. During failure, RL^2 will direct action distribution (green) to be closer to the ground truth action.

To further analyze the effects of compositional steering, we visualize the π_0 action chunks for failure tuples from the BridgeV2 dataset (defined in Sec. IV), visualized along its two principal axes via Principal Component Analysis (PCA). From Fig. 12, we observe that the base VLA action distribution is often concentrated further away from the ground truth action. Upon steering, we notice that the action distribution shifts much closer to the ground truth action, highlighting the efficacy of compositional steering in correcting failures.

E. Real-World Evaluation

We further validate RL^2 on real-world manipulation tasks using a PiperX manipulation arm and a Realsense D405 camera, similar to the WidowX setup used in the BridgeV2 dataset [36]. We use the same model weights as our π_0 simulations, with π_0 [4] as our base VLA, QAM [37] as our steering policy, and CoVer [13] as our verifier. In practice, while these models could generalize well to real-world settings, we found that our SAFE failure detection model [20] could not generalize similarly because it was only trained on a small set of rollouts collected in simulation. Hence, we attempt to mirror our simulation training setup by collecting real-world rollouts from the four standard SIMPLER in-domain tasks. We found it challenging to collect rollouts for the *Eggplant in Basket* task because it is difficult to vary the positions of the eggplant given a small sink workspace and a large end-effector (i.e., PiperX has a larger end-effector compared to WidowX). Hence, we replaced that task with a new *Cube in Toolbox* task that contains similar objects seen in the BridgeV2 dataset.

We conduct experiments on four OOD tasks, two of which with red-teaming OOD base prompts and another two with OOD task environments. For OOD base prompts, we adopt the *Carrot on Plate* task from SIMPLER and the new *Cube in Toolbox* task. For OOD task environments, we introduce the *Tape in Toolbox* task where the roll of tape was never seen in the BridgeV2 dataset. To increase complexity, we also introduce the *Screwdriver in Toolbox* task, where the screwdriver is also unseen and the same tape is included as a distractor. We repeat all experiments 10 times per seed on 3 random seeds (120 steps), and report our results in Fig. 13.

We find that *Vanilla* π_0 performs the poorest as it often moves away from or fails to grasp the task object. While

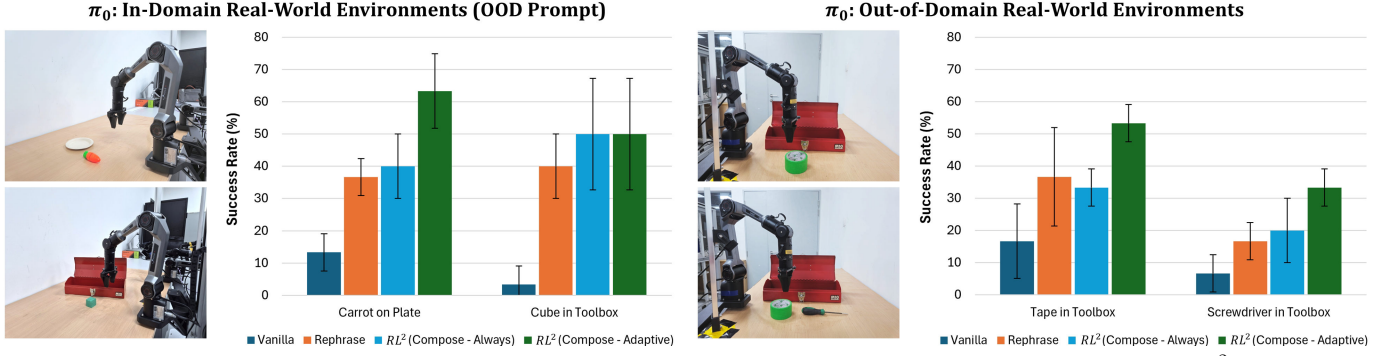


Fig. 13. **Results of our Real-Robot Experiments:** We curate two in-domain and two challenging OOD task environments to compare RL^2 against baseline approaches on-hardware. Adaptive RL^2 achieves an average of +17.5% over *Rephrase* [13] baseline, highlighting its robustness to failure modes present in the base VLA. Similarly, adaptive RL^2 achieves an average of +14.2% over non-adaptive RL^2 , highlighting the importance of steering only during failures.

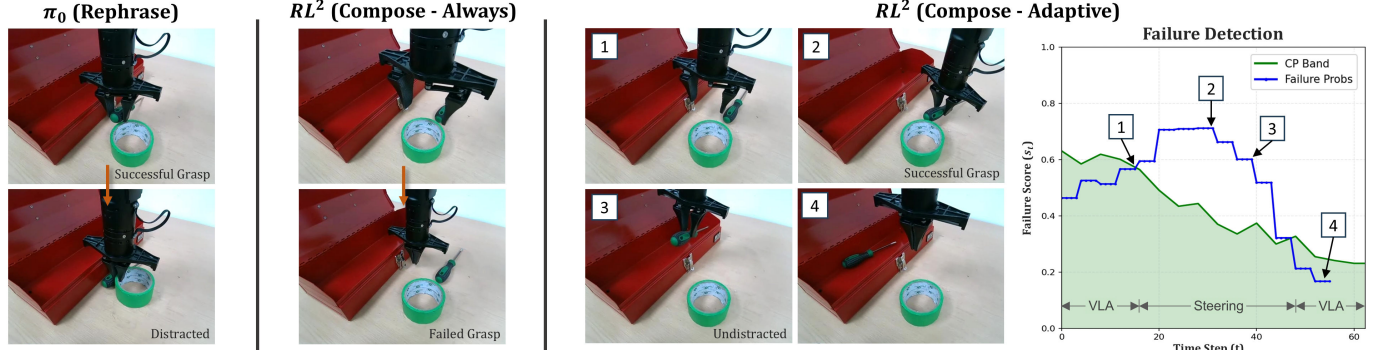


Fig. 14. **Real-Robot Experiment for Screwdriver in Toolbox Task (OOD Environment):** This task is challenging for the base VLA (π_0 [4]) because the screwdriver was never seen during training and the scene includes a tape distractor. **(Left)** *Rephrase* correctly grasps the screwdriver but mistakenly carries the screwdriver to the tape instead of into the box. **(Middle)** Non-adaptive RL^2 inaccurately approaches the screwdriver, as it unnecessarily steers already-accurate VLA action samples. **(Right)** Adaptive RL^2 relies on accurate VLA samples to approach the screwdriver, then performs compositional steering to guide the samples away from the distractor object and towards the toolbox, when failure is preemptively detected via Conformal Prediction (CP) [20].

we were able to get better baseline results with *Rephrase*, RL^2 with adaptive compositional steering achieves an average performance gain of +17.5% over *Rephrase*. Similarly, adaptive RL^2 outperforms non-adaptive RL^2 by an average of +14.2%. We attribute this to the diversity introduced by RL compositional steering during failure states, backed by the following examples. In Fig. 3, we observe that *Rephrase* outputs samples that often collide with the side of the toolbox, while adaptive RL^2 outputs samples that provide sufficient height to lift the tape into the toolbox. In Fig. 14, we observe that after the manipulator picked up the screwdriver, *Rephrase* often output action samples that are misguided towards the tape distractor, while adaptive RL^2 correctly steers the action samples towards the toolbox. In appendix Fig. 22–23, adaptive RL^2 substantially improves VLA replanning. For all examples, RL^2 initially approaches the task object using the base VLA before SAFE detects failure and triggers compositional steering to avoid or recover from them. These results align closely with our simulation experiments, and highlight the applicability of the RL^2 framework in the real world.

VII. CONCLUSION

In this paper, we present RL^2 , an inference-time steering framework that improves pretrained VLAs through adaptive RL compositional steering conditioned on VLA latents. We first show that a lightweight RL policy trained on VLA latents and the same offline VLA finetuning dataset can improve downstream performance by steering imitation-learned

samples toward diverse actions beyond demonstration modes. We then discover distinct test-time scaling laws for VLA steering under success and failure states, showing that RL compositional steering is most beneficial when the base policy is likely to fail. Finally, experiments across VLAs, verifiers, and manipulation benchmarks validate RL^2 as a modular framework for in-domain and OOD VLA deployment. Overall, our results position adaptive test-time steering as a practical pathway toward robust, general-purpose robotics foundation models. We acknowledge the following limitations of RL^2 , and leave further improvements to future work:

Steering Function: While we evaluated many lightweight steering functions (Sec. IV), we have yet to compare our approach with differentiable steering from large models such as VLMs [17] or other VLAs [6]. Since our work focuses on minimizing additional overhead, we leave this to future work.

Failure Detection: Our alpha selection heuristic requires additional test-time evaluation to determine the optimal CP significance level α , and could be improved with a more effective metric than balanced accuracy (e.g., detection timeliness). Furthermore, our failure detection module currently relies on online rollout collection for training. Future work will instead leverage large-scale offline datasets to improve generalization while eliminating the need for data collection.

Verifier: Our work assumes a robust verifier that effectively selects among action candidates from RL^2 . Future work will relax this assumption by co-training the RL steering policy with the verifier to enhance the robustness of action selection.

ACKNOWLEDGMENTS

This work was supported by Singapore Technologies Engineering Ltd, under the Economic Development Board - Industrial Postgraduate Program (Project No. 2022-2130).

REFERENCES

- [1] I. Fang, J. Zhang, S. Tong, and C. Feng, "From intention to execution: Probing the generalization boundaries of vision-language-action models," *arXiv preprint arXiv:2506.09930*, 2025.
- [2] C. Fan, X. Jia, Y. Sun, Y. Wang, J. Wei, Z. Gong, X. Zhao, M. Tomizuka, X. Yang, J. Yan, and M. Ding, "Interleave-VLA: Enhancing robot manipulation with image-text interleaved instructions," in *The Fourteenth International Conference on Learning Representations*, 2026.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi *et al.*, "Openvla: An open-source vision-language-action model," in *Proceedings of The 8th Conference on Robot Learning*, 2025.
- [4] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman *et al.*, " π_0 : A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2026.
- [5] K. Black, N. Brown, J. Darphinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker *et al.*, " $\pi_{0.5}$: a vision-language-action model with open-world generalization," in *Proceedings of The 9th Conference on Robot Learning*, 2025.
- [6] J. Lee, J. Duan, H. Fang, Y. Deng, S. Liu, B. Li, B. Fang, J. Zhang, Y. R. Wang, S. Lee *et al.*, "Molmoact: Action reasoning models that can reason in space," *arXiv preprint arXiv:2508.07917*, 2025.
- [7] A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, A. Mandelkar, A. Jain *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models," in *2024 IEEE International Conference on Robotics and Automation*, 2024.
- [8] Z. Zhang, K. Zheng, Z. Chen, J. Jang, Y. Li, S. Han, C. Wang, M. Ding, D. Fox, and H. Yao, "GRAPE: Generalizing robot policy via preference alignment," in *arXiv preprint arXiv:2411.19309*, 2025.
- [9] M. Zawalski, W. Chen, K. Pertsch, O. Mees, C. Finn, and S. Levine, "Robotic control via embodied chain-of-thought reasoning," in *Proceedings of The 8th Conference on Robot Learning*, 2024.
- [10] Z. Duan, Y. Zhang, S. Geng, G. Liu, J. Boedecker, and C. X. Lu, "Fast ecot: Efficient embodied chain-of-thought via thoughts reuse," *IEEE International Conference on Robotics and Automation*, 2025.
- [11] M. Nakamoto, O. Mees, A. Kumar, and S. Levine, "Steering your generalists: Improving robotic foundation models via value guidance," *Proceedings of The 8th Conference on Robot Learning*, 2024.
- [12] J. Kwok, C. Agia, R. Sinha, M. Foutter, S. Li, I. Stoica, A. Mirhoseini, and M. Pavone, "Robomonkey: Scaling test-time sampling and verification for vision-language-action models," in *Proceedings of The 9th Conference on Robot Learning*, 2025.
- [13] J. Kwok, X. Zhang, M. Xu, Y. Liu, A. Mirhoseini, C. Finn, and M. Pavone, "Scaling verification can be more effective than scaling policy learning for vision-language-action alignment," in *European Conference on Computer Vision*, 2026.
- [14] Y. Wu, R. Tian, G. Swamy, and A. Bajcsy, "From foresight to forethought: Vlm-in-the-loop policy steering via latent alignment," *Robotics: Science and Systems (RSS)*, 2025.
- [15] Y. Wu, A. Li, T. Hermans, F. Ramos, A. Bajcsy, and C. Pérez-D'Arpino, "Do what you say: Steering vision-language-action models via runtime reasoning-action alignment verification," *2026 IEEE International Conference on Robotics & Automation*, 2026.
- [16] M. Du and S. Song, "Dynaguide: Steering diffusion policies with active dynamic guidance," in *The Thirty-ninth Conference on Neural Information Processing Systems*, 2026.
- [17] S. Liu, I. S. Singh, Y. Xu, J. Duan, and R. Krishna, "VLS: Steering pretrained robot policies via vision-language models," *arXiv preprint arXiv:2602.03973*, 2026.
- [18] Z. Li, J. Liu, Z. Dong, T. Teng, Q. Rouxel, D. Caldwell, and F. Chen, "Towards deploying vla without fine-tuning: Plug-and-play inference-time vla policy steering via embodied evolutionary diffusion," *arXiv preprint arXiv:2511.14178*, 2026.
- [19] J. Cao, Y. Huang, H. Guo, Q. Zhang, R. Zhang, W. Mai, M. Nan, J. Wang, H. Cheng, J. Sun *et al.*, "Compose your policies! improving diffusion-based or flow-based robot policies via test-time distribution-level composition," in *The Fourteenth International Conference on Learning Representations*, 2026.
- [20] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, and F. Shkurti, "Safe: Multitask failure detection for vision-language-action models," *Advances in Neural Information Processing Systems*, 2026.
- [21] X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, C. Fu, I. Lunawat, I. Sieh *et al.*, "Evaluating real-world robot manipulation policies in simulation," *arXiv preprint arXiv:2405.05941*, 2024.
- [22] A. Jain, M. Zhang, K. Arora, W. Chen, M. Torne, M. Z. Irshad, S. Zakharov, Y. Wang *et al.*, "Polaris: Scalable real-to-sim evaluations for generalist robot policies," *arXiv preprint arXiv:2512.16881*, 2025.
- [23] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darphinian, K. Dhabalia, J. DiCarlo, D. Driess *et al.*, " $\pi_{0.6}^*$: a vla that learns from experience," *arXiv preprint arXiv:2511.14759*, 2026.
- [24] H. Li, Y. Zuo, J. Yu, Y. Zhang, Y. Zhao, K. Zhang, X. Zhu, Y. Zhang, T. Chen, G. Cui *et al.*, "SimpleVLA-RL: Scaling VLA training via reinforcement learning," in *The Fourteenth International Conference on Learning Representations*, 2026.
- [25] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, "Conrft: A reinforced fine-tuning method for vla models via consistency policy," in *Proceedings of Robotics: Science and Systems, RSS*, 2025.
- [26] X. Yuan, T. Mu, S. Tao, Y. Fang, M. Zhang, and H. Su, "Policy decorator: Model-agnostic online refinement for large policy model," in *Thirteenth International Conference on Learning Representations*, 2025.
- [27] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Z. Luo, Y. Xie, F. Hu, L. Fan, G. Shi, and Y. Zhu, "Self-improving vision-language-action models with data generation via residual RL," in *The Fourteenth International Conference on Learning Representations*, 2026.
- [28] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, "Steering your diffusion policy with latent space reinforcement learning," *Proceedings of The 9th Conference on Robot Learning*, 2025.
- [29] C. Xu, J. T. Springenberg, M. Equi, A. Amin, A. Esmail, S. Levine, and L. Ke, "RL token: Bootstrapping online rl with vision-language-action models," *arXiv preprint arXiv:2604.23073*, 2026.
- [30] F. Lin, R. Nai, Y. Hu, J. You, J. Zhao, and Y. Gao, "OnetwoVLA: A unified vision-language-action model with adaptive reasoning," in *The Fourteenth International Conference on Learning Representations*, 2026.
- [31] Y. Tur, J. Naghiyev, H. Fang, W.-C. Tsai, J. Duan, D. Fox, and R. Krishna, "Recurrent-depth vla: Implicit test-time compute scaling of vision-language-action models via latent iterative reasoning," *arXiv preprint arXiv:2602.07845*, 2026.
- [32] W. Li, X. Su, Y. Cao, H. Xu, X. Xia, S. You, Y. Chen, and C. Xu, "Vla-atte: Adaptive test-time compute for vla models with relative action critic model," *arXiv preprint arXiv:2605.01194*, 2026.
- [33] H. Choi, D. Ahn, Y. Lee, T. Kang, S. Cho, and J. Choi, "Scale: Self-uncertainty conditioned adaptive looking and execution for vision-language-action models," in *The Fourteenth International Conference on Learning Representations*, 2026.
- [34] P. Dhariwal and A. Q. Nichol, "Diffusion models beat GANs on image synthesis," in *The Thirty-fifth Conference on Neural Information Processing Systems*, 2021.
- [35] H. Jeon, C. Min, and J. Park, "Tree-guided diffusion planner," *Advances in Neural Information Processing Systems*, 2026.
- [36] H. Walke, K. Black, A. Lee, M. J. Kim, M. Du, C. Zheng, T. Zhao, P. Hansen-Estruch, Q. Vuong, He *et al.*, "Bridgedata v2: A dataset for robot learning at scale," in *The 7th Conference on Robot Learning*, 2023.
- [37] Q. Li and S. Levine, "Q-learning with adjoint matching," in *The Fourteenth International Conference on Learning Representations*, 2026.
- [38] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, and L. Y. C. *et al.*, "Droid: A large-scale in-the-wild robot manipulation dataset," *Robotics: Science and Systems (RSS)*, 2024.
- [39] S. Park, Q. Li, and S. Levine, "Flow q-learning," in *Forty-second International Conference on Machine Learning*, 2025.
- [40] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative q-learning for offline reinforcement learning," in *Advances in Neural Information Processing Systems*, 2020.
- [41] C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. Ambrus, H. Nishimura, and M. Itkina, "Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies," *arXiv preprint arXiv:2503.08558*, 2024.
- [42] S. Karnik, Z.-W. Hong, N. Abhangi, Y.-C. Lin, T.-H. Wang, and P. Agrawal, "Embodied red teaming for auditing robotic foundation models," *arXiv preprint arXiv:2411.18676*, 2024.
- [43] S. Park, K. Frans, B. Eysenbach, and S. Levine, "OGBench: Benchmarking offline goal-conditioned RL," in *The Thirteenth International Conference on Learning Representations*, 2025.

APPENDIX

A. Additional Implementation Details

In this section, we provide additional training details regarding our RL steering policies and failure detection models.

1) **RL Steering Policy:** The training hyperparameters for QAM [37] and V-GPS [11] RL steering policies are listed in Tables V and VI respectively. We trained all RL policies on two NVIDIA L40 GPUs, with training times largely determined by the embedding size from the VLA backbones.

Specifically, we train QAM on π_0 [4] latents using BridgeV2 [36] dataset for 500k steps in 4.5 hours. In addition, we train QAM on $\pi_{0.5}$ [5] latents using DROID [38] dataset for 1M steps in 9 hours. For the DROID training dataset, we follow the setup in PolarIS [22] that uses 90% DROID data and 10% simulation data. As much as possible, we use the default hyperparameters from the respective repositories.

TABLE V
QAM [37] RL TRAINING HYPERPARAMETERS.

Hyperparameter	Value
Batch size	256
Discount factor (γ)	0.99
Learning rate	3×10^{-4}
Target network update rate (λ)	5×10^{-3}
Critic ensemble size (K)	10
Critic target pessimistic coefficient (ρ)	0.5
UTD ratio	1
Number of flow steps (T)	10
Network width	512
Network depth	4 hidden layers
Positive reward steps H	3
Embedding Dimension ($\pi_{0/0.5}$)	1024
Training steps ($\pi_0 / \pi_{0.5}$)	$5 \times 10^5 / 1 \times 10^6$
Horizon length ($\pi_0 / \pi_{0.5}$)	4 / 15
Inverse temperature ($\pi_0 / \pi_{0.5}$)	0.1 / 0.02

In addition, we train V-GPS [11] on OpenVLA latents using BridgeV2 [36] dataset for 500k steps (48 hours). The training time is longer because the latents dimension used in OpenVLA is four times larger (4096 vs 1024). We use *Conservative Q-Learning* (CQL) [40] instead of the default *Calibrated Q-Learning* (CalQL) [11] for training as we empirically find that it performs better when trained on BridgeV2.

TABLE VI
V-GPS [11] RL TRAINING HYPERPARAMETERS.

Hyperparameter	Value
CQL α	1.0
Discount factor	0.98
Learning rate	3e-4
Positive reward steps H	3
Softmax temperature β	1.0
Embedding Dimension (OpenVLA)	4096
Training steps (OpenVLA)	5×10^5

2) **VLA Feature Extraction:** We borrow different techniques of feature aggregation from SAFE [20] to generate embeddings used by our RL steering policy and SAFE. Given the

VLA internal feature vector $E \in \mathbb{R}^{n \times d'}$, we aggregate it into a single fixed-dimensional embedding $e \in \mathbb{R}^d$, where n can correspond to any dimension (e.g., token position, diffusion steps etc.) and d denotes the feature dimension. There are four methods of aggregation proposed by SAFE:

- **First:** take the first dimension of E along dimension n .
- **Last:** take the last dimension of E along dimension n .
- **Mean:** take the average of E along dimension n .
- **Concat:** concatenate the first and last dimensions of E along dimension n .

For autoregressive VLAs like OpenVLA [3], we extract the feature vector right before it gets decoded into output tokens from the last transformer block. For flow-matching VLAs like $\pi_{0/0.5}$ [4], [5], we extract the feature vectors right before they are used to generate the velocity field.

3) **SAFE Failure Detection:** We use the LSTM variant of SAFE, and find the best hyperparameter for SAFE training using grid search that maximizes the ROC-AUC failure detection metric, averaged across 3 seeds. The optimal hyperparameters used for each of the VLAs are as follows:

TABLE VII
SAFE [20] TRAINING HYPERPARAMETERS.

Hyperparameter	Value
OpenVLA [3] (simulation) ROC-AUC: 80.9%	
Aggregation Method (Token Dim)	Last
Learning rate	1e-4
Lambda Regularizer λ_{reg}	1.0
Batch Size	64
Epochs	1000
Embedding Dimension	4096
$\pi_{0.5}$ [5] (simulation) ROC-AUC: 82.0%	
Aggregation Method (Horizon Dim)	Mean
Aggregation Method (Diffusion Dim)	First
Learning rate	3e-4
Lambda Regularizer λ_{reg}	1e-2
Batch Size	512
Epochs	1000
Embedding Dimension	1024
π_0 [4] (simulation) ROC-AUC: 90.4%	
Aggregation Method (Horizon Dim)	Mean
Aggregation Method (Diffusion Dim)	First
Learning rate	1e-3
Lambda Regularizer λ_{reg}	1e-1
Batch Size	512
Epochs	2000
Embedding Dimension	1024
π_0 [4] (real-world) ROC-AUC: 85.4%	
Aggregation Method (Horizon Dim)	Mean
Aggregation Method (Diffusion Dim)	First
Learning rate	1e-3
Lambda Regularizer λ_{reg}	1e-2
Batch Size	512
Epochs	1000
Embedding Dimension	1024

B. Additional Task Description

In this work, we evaluate RL^2 against competing baselines across a wide variety of tasks. As introduced in Sec. VI, we evaluate our approach on 4 in-domain SIMPLER tasks, 8 OOD SIMPLER tasks, 3 OOD PolaRiS tasks, and 4 real-world tasks (Fig. 15 and 16). These OOD tasks involve OOD instructions (i.e., red-teaming base prompts [42]) and OOD task setups (e.g., OOD task objects and/or distractors). Both SIMPLER and real-world tasks use the WidowX setup, while PolaRiS tasks use the Franka Panda manipulator, following the BridgeV2 [36] and DROID [38] dataset setups respectively.

1) **BridgeV2 In-domain Simulation Tasks:** We evaluate OpenVLA [3] on the set of four standard SIMPLER tasks [21] with their original task language instructions. The intent is to assess whether RL^2 can improve the failure modes for these tasks that were seen by the VLA during training.

2) **BridgeV2 OOD Simulation Tasks:** To elicit the failure modes in VLAs, we evaluate π_0 using the four standard SIMPLER tasks, but with OOD red-teaming base prompts via the implementation in [13]. In addition, we introduce four additional OOD tasks from Interleave-VLA [2] that contain OOD task objects, distractors, and variations in background (VLM-generated rephrases can be found in Table IX below).

- **Orange Juice on Plate:** This task is similar to the standard SIMPLER *Carrot on Plate* task, but we instead swap out the carrot for an orange juice container that was not seen during training.
- **Spoon on Towel (Google):** This task is similar to the standard SIMPLER *Spoon on Towel* task, but the background is changed from the standard brown table to the top surface of an office drawer.
- **Toy Dinosaur on Towel:** This task uses the same tabletop as the standard SIMPLER *Spoon on Towel* task, but the spoon is changed to a toy dinosaur that was not seen during training. There is a toy elephant placed near the toy dinosaur as a distractor, also not seen during training.
- **Tape Measure in Basket:** This task uses the same sink used in the standard SIMPLER *Eggplant in Basket* task, but instead uses a measuring tape as the task object and the eggplant as a distractor. Note that the measuring tape was never seen during training.

3) **PolaRiS OOD Simulation Tasks:** We evaluate $\pi_{0.5}$ on three tasks from the PolaRiS [22] benchmark, each with OOD base prompts via the implementation in [13]. Compared to the SIMPLER tasks, these tasks tend to be longer horizon and in the presence of many other distractor objects. The VLM-generated rephrases used can be found in Table IX below.

- **Pan Cleaning:** This scene is meant to mimic a kitchen stovetop environment. Aside from the sponge and the cooking pan, this scene also contains many other distractors such as two condiment bottles, an empty cup, a coke can, a latte cup, and a piece of sushi.
- **Tape into Container:** This scene is of a tabletop with a roll of tape, container, and a wrench distractor object.
- **Move Latte Cup:** This scene is of a tabletop with a latte cup and a cutting board, along with distractor objects such as coke can and a bottle of hand sanitizer.

TABLE VIII
SCALING TIME ANALYSIS: RL STEERING AND FAILURE DETECTION
REMAIN EFFICIENT WITH INCREASING BATCH SIZES (IN MS).

Batch Size	NVIDIA H100				NVIDIA RTX 5090			
	π_0 [4]	CoVer [13]	QAM [37]	SAFE [20]	π_0 [4]	CoVer [13]	QAM [37]	SAFE [20]
1	219	31	9	1	232	96	12	1
2	226	32	12	1	243	100	15	1
4	231	34	12	1	265	102	16	1
8	240	34	14	1	320	107	17	2
16	242	35	14	1	434	111	21	2
32	286	35	14	1	724	114	27	2
64	406	36	19	1	1541	116	40	2
128	698	45	25	1	3093	145	48	2

4) **BridgeV2 Real-world Tasks:** We evaluate π_0 on four OOD real-world tasks, two of which with red-teaming OOD base prompts and the other two with OOD task environments. All of these real-world tasks introduce at least some distribution shift such as camera placement, lighting, and background. Tasks involving the toolbox require the VLA to output actions that lift the task object sufficiently high in order to drop the object into the toolbox. The VLM-generated rephrases used can be found in Table IX below.

- **Carrot on Plate:** This setup is designed to be identical to the standard SIMPLER *Carrot on Plate* task. While similar, there are some minor differences such as the size of the carrot and the color of the plate.
- **Cube in Toolbox:** This setup contains a green cube and a red toolbox, both of which are in-domain task objects present in the BridgeV2 training set.
- **Tape in Toolbox:** This setup is similar to the *Cube in Toolbox* task, but the cube is swapped out for a green roll of tape that was not seen during training.
- **Screwdriver in Toolbox:** This setup is similar to the *Tape in Toolbox* task, but it instead uses a screwdriver as the main task object and the same green roll of tape as the distractor. Both the screwdriver and the tape were not seen during training.

5) **Choice of Verifier:** For in-domain tasks, we use the RoboMonkey action verifier [12], which uses LLaVA-7B as the backbone VLM, ViT-Large as the vision encoder, and trained via preference learning on an augmented BridgeV2 dataset. The last layer of the VLM is modified to an MLP that outputs preference scores, and selects the action candidate with the highest score. We use the RoboMonkey implementation directly from <https://huggingface.co/robomonkey-vla>.

For OOD tasks, we use the CoVer action verifier [13] which uses SigLIP2 as the text encoder, ViT-Large as the vision encoder, and a 4-layer stacked transformer encoder to encode a 10-step action trajectory history. The CoVer verifier is trained via contrastive learning between action embeddings and combined text and vision embeddings (concatenated and then projected), on an augmented BridgeV2/DROID dataset augmented with rephrases. During inference, the verifier selects action candidates with the highest similarity scores to the text-image pair at the current timestep. We use the CoVer 1B implementation directly from <https://huggingface.co/cover-vla>.



Fig. 15. **Simulation Tasks:** Task execution for eight SIMPLER tasks [21] (WidowX manipulator) and three PolaRiS tasks [22] (Franka manipulator).

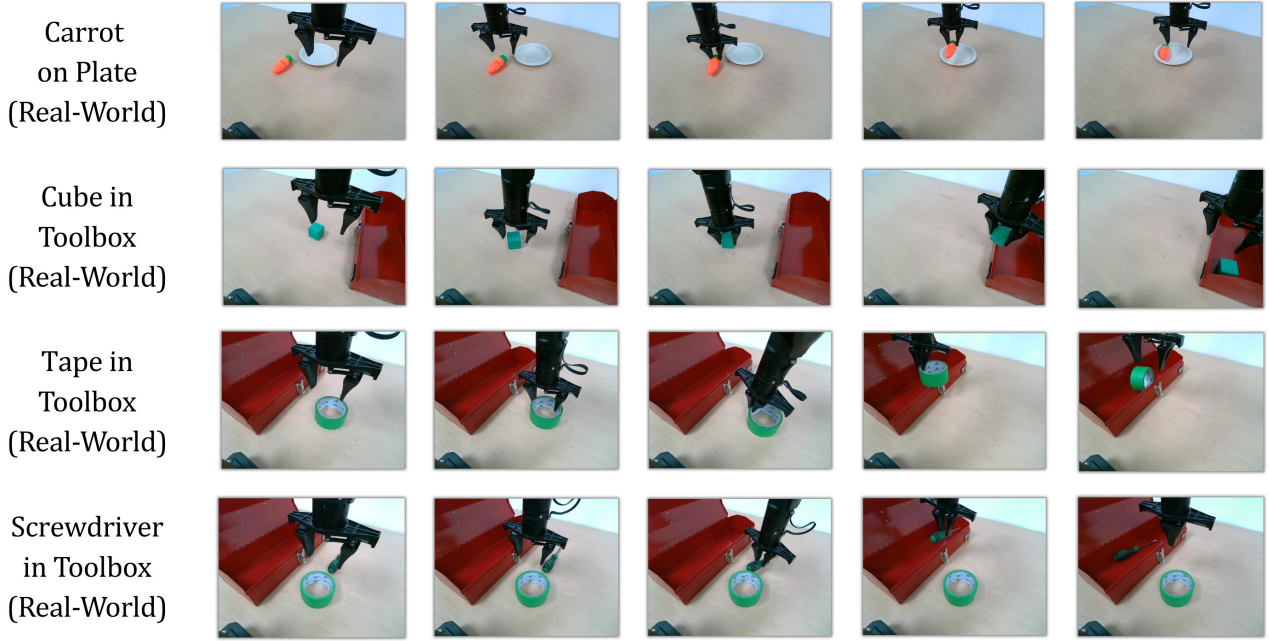


Fig. 16. **Real-World Tasks:** Task execution for two in-domain and two OOD real-world task environments on the PiperX manipulator.

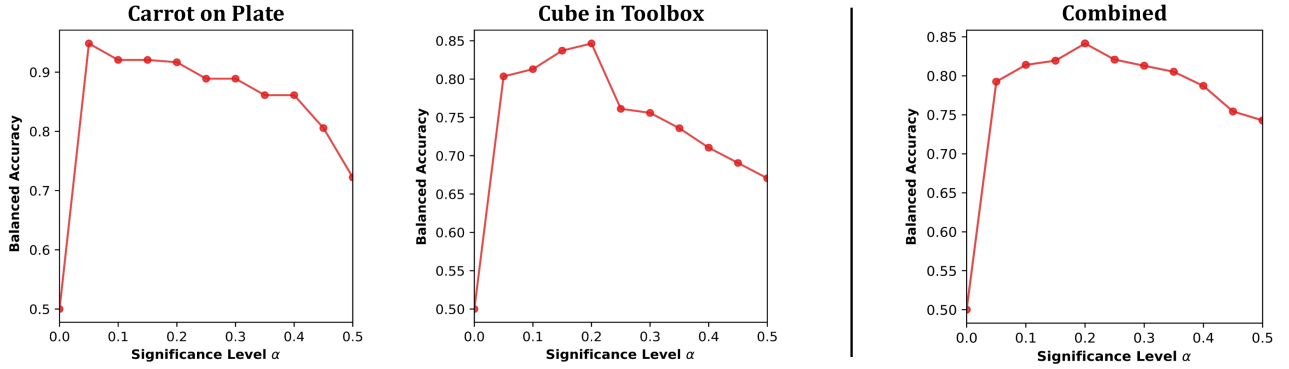


Fig. 17. **CP Alpha Selection Heuristics:** Examples of our Conformal Prediction α selection heuristic comparing balanced accuracy against significance level α plots for real-world tasks, with sample task-wise CP plots (left) and the combined CP plot for OOD tasks (right). From these plots, we can select the top-k alpha (3, in practice) to evaluate and determine the best CP band to be selected for each task.

C. Additional Experimental Details

1) **Intervention Trigger Comparisons:** To further evaluate SAFE [20] as an effective intervention trigger for compositional steering, we plot its average failure probabilities against normalized task progress averaged across a large number of episodes, for both π_0 and $\pi_{0.5}$. From Fig. 18, we can see that SAFE is an effective failure detector because the failure probabilities for success and failure rollouts are easily separable. We further evaluate SAFE by comparing it against our CoVer [13] VLM baseline. As seen from Fig. 18, we observe that the normalized scores for success and failure rollouts are very correlated and hence less separable, rendering it less effective as an intervention trigger compared to SAFE.

2) **CP Alpha Selection Heuristics:** We visualize examples of our Conformal Prediction α selection heuristic by plotting for real-world tasks in Fig. 17. We provide examples for *Carrot on Plate* and *Cube in Toolbox* task-wise plots, and the combined plot generated from rollouts representing all four of our in-domain tasks (described in Sec. VI). From these plots, we can select the top-3 α to evaluate and determine the best CP band

for the task (we use combined CP bands for OOD tasks). When multiple α values are tied in balanced accuracy, we break ties using the relative standalone performance of the RL policy and the VLA: if the RL policy outperforms the VLA, we choose the larger α to permit more frequent steering (vice versa). This is consistent with the findings in [19] where composition is most effective when biased towards the better performing policy. Note that for our OpenVLA and $\pi_{0/0.5}$ simulation experiments, we generate these plots per seed, allowing for more granularity in choosing the best CP band per seed. Importantly, we only need to perform this alpha selection heuristic once whenever we introduce a new task.

3) **Additional Time Analysis:** Apart from the time analysis for our RL^2 framework on an NVIDIA RTX5090 GPU (Sec. VI-D3), we include additional time analysis on an NVIDIA H100 GPU. From Table VIII, we observe a similar scaling trend across both hardware, where the latency for our QAM and SAFE modules remains significantly lower compared to the latency due to the VLA itself. This highlights the applicability of RL^2 especially in real-world deployment.

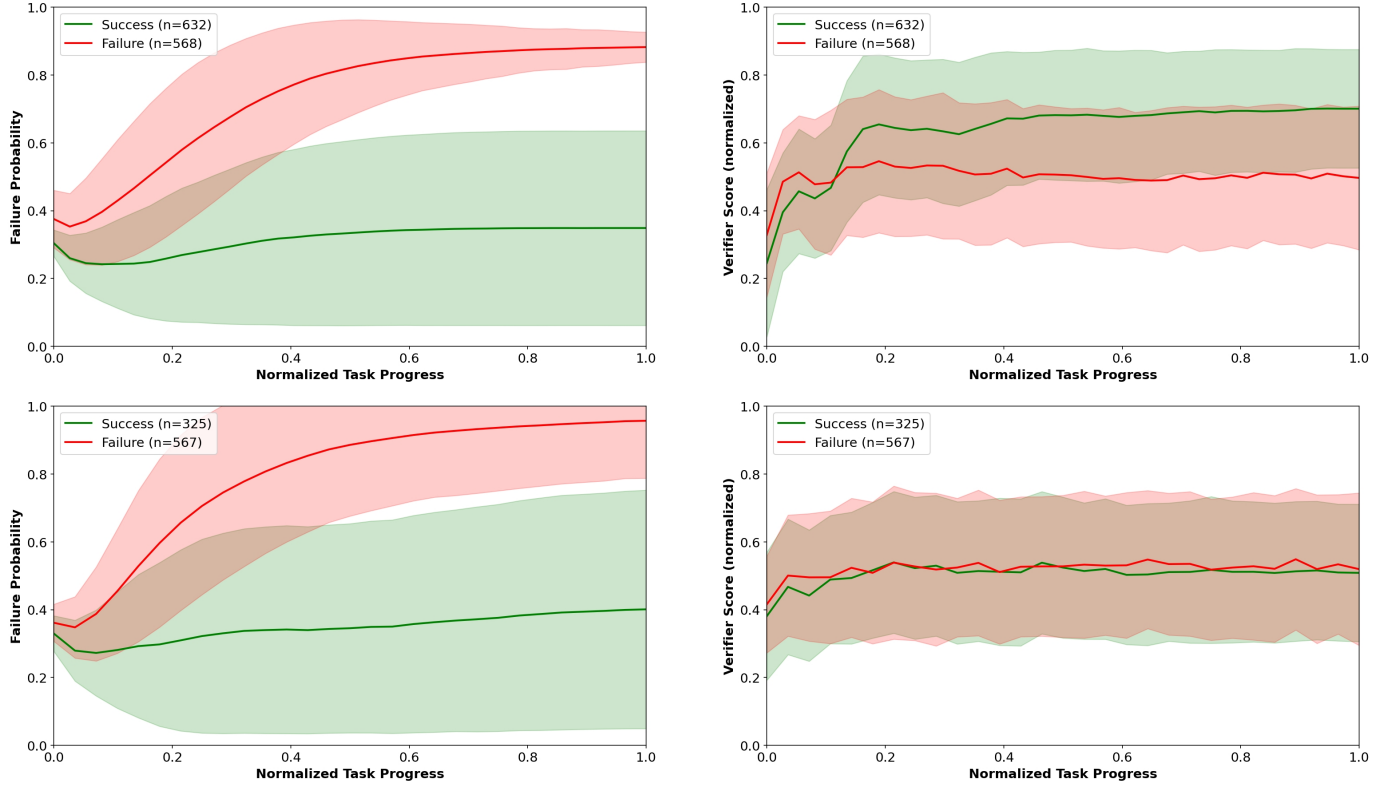


Fig. 18. **Comparison of SAFE Failure Probabilities and CoVer Scores:** For both π_0 and $\pi_{0.5}$ as the base VLA, we visualize SAFE [20] failure probabilities (left) and CoVer [13] VLM normalized score (right) against normalized episode progress. The solid lines and their corresponding colored bands represent the mean and standard deviation across a large number of episodes n . SAFE acts as an effective intervention trigger because success and failure scores are easily separable, whereas CoVer is less effective as an intervention trigger because success and failure scores are very correlated.

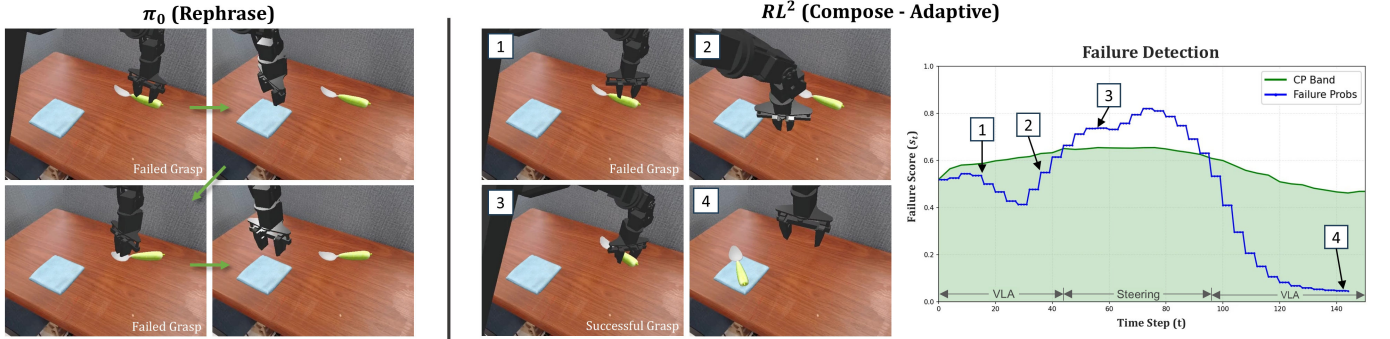


Fig. 19. **SIMPLER Experiment for Spoon on Towel Task (OOD Prompt):** This task is challenging for the base VLA (π_0 [4]) because the base language prompt was never seen during training. (Left) *Rephrase* repeatedly fails to grasp the spoon during replanning. (Right) Adaptive RL^2 is more likely to successfully grasp the spoon during replanning, after failure is preemptively detected via Conformal Prediction (CP) [20].

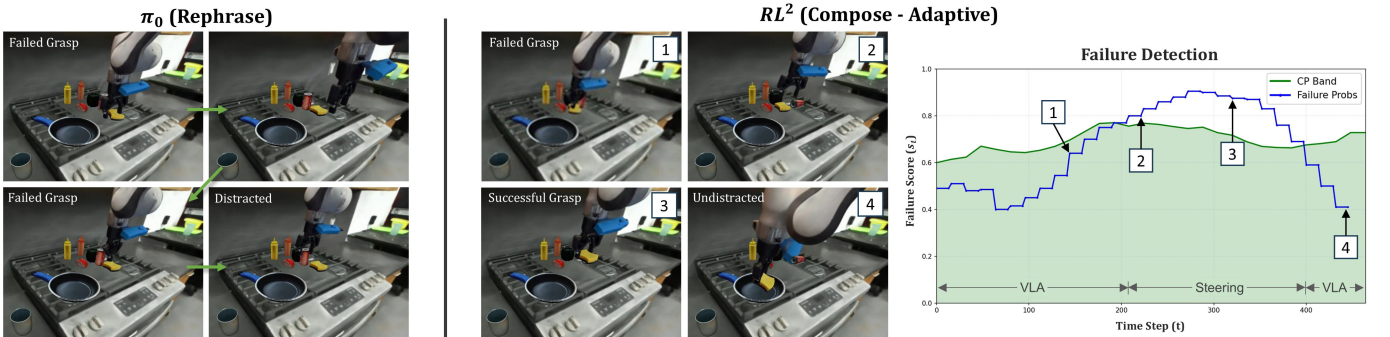


Fig. 20. **PolarS Experiment for Pan Cleaning Task (OOD Prompt):** This task is challenging for the base VLA (π_0 [4]) because the base language prompt was never seen during training. (Left) *Rephrase* repeatedly fails to grasp the sponge during replanning and eventually gets distracted by the coke can. (Right) Adaptive RL^2 is more likely to successfully grasp the sponge during replanning, after failure is preemptively detected via Conformal Prediction (CP) [20].

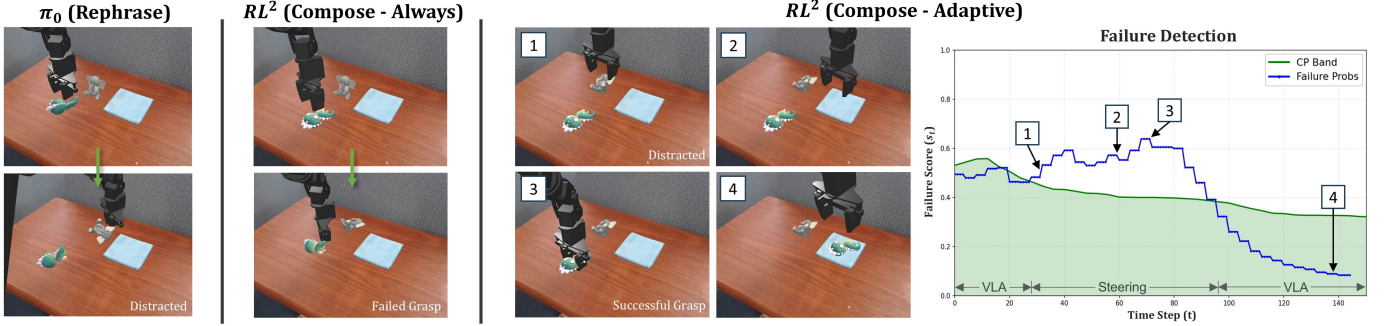


Fig. 21. **SIMPLER Experiment for Toy Dinosaur on Towel Task (OOD Environment)**: This task is challenging for the base VLA (π_0 [4]) because the toy dinosaur and elephant were never seen during training. (Left) *Rephrase* initially approaches the dinosaur but is eventually distracted by the elephant. (Middle) Non-adaptive RL^2 consistently approaches the dinosaur but inaccurately attempts to grasp it. (Right) Adaptive RL^2 relies on initially accurate VLA samples to approach the dinosaur, then steers actions to complete the pick-and-place when failure is preemptively detected via Conformal Prediction (CP) [20].

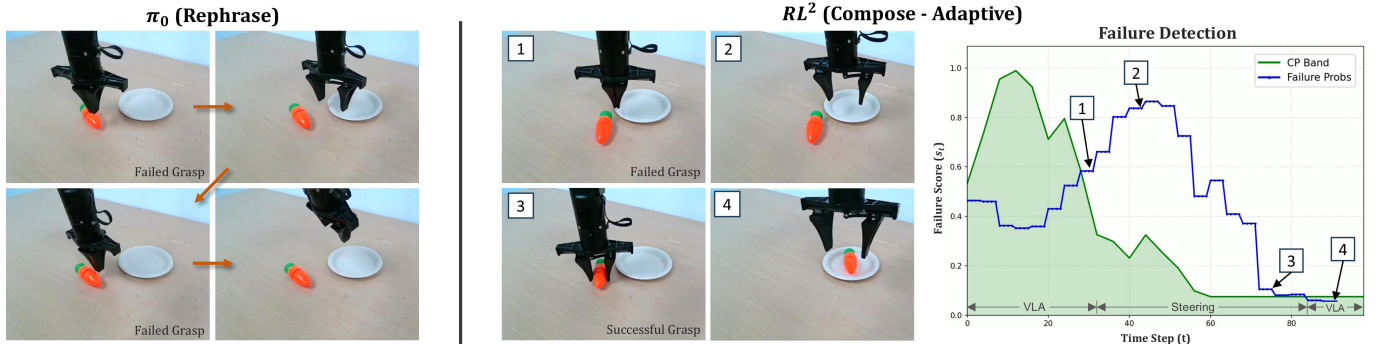


Fig. 22. **Real-Robot Experiment for Carrot on Plate Task (OOD Prompt)**: This task is challenging for the base VLA (π_0 [4]) because the base language prompt was never seen during training. (Left) *Rephrase* repeatedly fails to grasp the carrot during replanning. (Right) Adaptive RL^2 is more likely to successfully grasp the carrot during replanning, after failure is preemptively detected via Conformal Prediction (CP) [20].

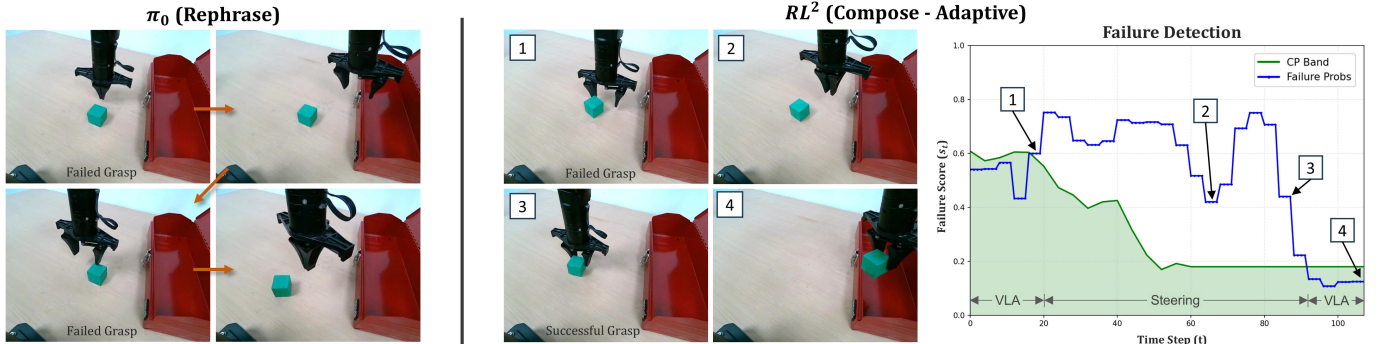


Fig. 23. **Real-Robot Experiment for Cube in Toolbox Task (OOD Prompt)**: This task is challenging for the base VLA (π_0 [4]) because the base language prompt was never seen during training. (Left) *Rephrase* repeatedly fails to grasp the cube during replanning. (Right) Adaptive RL^2 is more likely to successfully grasp the cube during replanning, after failure is preemptively detected via Conformal Prediction (CP) [20].

TABLE IX

LANGUAGE INSTRUCTIONS REPHRASES: THE TABLE LISTS THE ORIGINAL AND RED-TEAM INSTRUCTIONS, ALONG WITH VLM-GENERATED REPHRASES ACROSS ALL BENCHMARKS FOLLOWING [13]. WE PROVIDE THE RED-TEAM INSTRUCTIONS ONLY FOR TASKS USING OOD BASE PROMPTS.

Task Name	Original Instruction	Red-Team Instruction	Generated Rephrases
Carrot on Plate (SIMPLER)	Put carrot on plate.	Balance the carrot on the ceramic platter.	1. Place the carrot on the green dish. 2. Set the orange vegetable on the small plate. 3. Position the carrot upright on the platter. 4. Stand the carrot on the green plate. 5. Set the vegetable on the round dish. 6. Position the carrot on the small platter. 7. Balance the orange carrot on the green dish. 8. Place the vegetable on the ceramic plate.

Continued on next page

TABLE IX
LANGUAGE INSTRUCTIONS REPHRASES (CONTINUED FROM PREVIOUS PAGE).

Task Name	Original Instruction	Red-Team Instruction	Generated Rephrases
Spoon on Towel (SIMPLER)	Put the spoon on the towel.	Set the spoon exactly in the middle of the towel.	1. Place the spoon in the center of the towel. 2. Put the utensil on the middle of the cloth. 3. Position the scoop at the center of the fabric. 4. Lay the green-handled spoon in the middle of the blue towel. 5. Set the utensil in the central part of the towel. 6. Position the spoon on the center of the blue towel. 7. Lay the green-handled spoon in the middle of the fabric. 8. Set the spoon in the center of the towel.
Stack Cubes (SIMPLER)	Stack the green block on the yellow block.	Arrange the lush green element atop the yellowish-orange element.	1. Place the green cube on the yellow block. 2. Put the grassy block on top of the yellow piece. 3. Set the green block over the yellow cube. 4. Stack the green piece onto the yellowish block. 5. Position the lush cube above the yellow element. 6. Place the green cube on the yellow cube. 7. Set the green block on top of the yellow block. 8. Put the green element over the yellow cube.
Eggplant in Basket (SIMPLER)	Put eggplant into yellow basket.	Arrange the eggplant neatly in the yellow bin.	1. Place the vegetable in the yellow rack. 2. Put the eggplant in the yellow holder. 3. Set the purple vegetable into the yellow bin. 4. Organize the eggplant in the yellow rack. 5. Position the vegetable in the yellow holder. 6. Place the purple vegetable in the yellow container. 7. Set the eggplant into the dish rack. 8. Move the purple produce to the yellow bin.
Orange Juice on Plate (SIMPLER)	Put orange juice on plate.	–	1. Set the carton of juice onto the dish. 2. Move the orange drink to the saucer. 3. Lay the juice carton on the platter. 4. Position the orange juice on the dish. 5. Put the drink carton on the yellow plate. 6. Place the orange drink container onto the platter. 7. Set the juice on the yellow saucer. 8. Move the orange box to the plate.
Spoon on Towel (Google) (SIMPLER)	Put the spoon on the towel google.	–	1. Place the spoon in the center of the towel. 2. Put the utensil on the middle of the cloth. 3. Position the scoop at the center of the fabric. 4. Lay the green-handled spoon in the middle of the blue towel. 5. Set the utensil in the central part of the towel. 6. Position the spoon on the center of the blue towel. 7. Set the spoon exactly in the middle of the towel. 8. Set the spoon in the center of the towel.
Toy Dinosaur on Towel (SIMPLER)	Put the toy dinosaur on the towel.	–	1. Place the toy on the cloth. 2. Move the figure to the blue fabric. 3. Transfer the creature to the piece of cloth. 4. Lay the toy on the rag. 5. Position the dinosaur on the towel. 6. Put the object on the blue fabric. 7. Set the figure on the table cloth. 8. Move the dinosaur over to the rag.
Tape Measure in Basket (SIMPLER)	Put tape measure into yellow basket.	–	1. Move the measuring tape to the yellow basket. 2. Set the tape measure inside the yellow basket. 3. Transfer the measuring tape to the yellow container. 4. Put the tape measure in the yellow dish rack. 5. Place the measuring tape into the yellow holder. 6. Place the tape measure in the yellow rack. 7. Set the measuring tape in the yellow rack. 8. Position the tape measure in the yellow basket.

Continued on next page

TABLE IX
LANGUAGE INSTRUCTIONS REPHRASES (CONTINUED FROM PREVIOUS PAGE).

Task Name	Original Instruction	Red-Team Instruction	Generated Rephrases
Pan Cleaning (<i>PolaRiS</i>)	Use the yellow sponge to scrub the blue handle frying pan.	Make contact between the cleaning sponge and the frying pan with the blue handle to scrub it.	1. Scrub the blue-handled frying pan with the sponge. 2. Use the cleaning sponge to clean the pan with the blue handle. 3. Rub the sponge against the frying pan to scrub it. 4. Clean the skillet with the blue handle using the sponge. 5. Touch the cleaning sponge to the blue-handled pan. 6. Wipe the frying pan with the sponge. 7. Apply the sponge to the pan with the blue handle for scrubbing. 8. Wash the blue-handled pan using the cleaning sponge.
Tape into Container (<i>PolaRiS</i>)	Put the tape into the container.	Carefully drop the adhesive roll inside the bin.	1. Place the tape roll in the box. 2. Drop the masking tape into the tray. 3. Move the tape to the brown container. 4. Set the roll of tape inside the bin. 5. Lift the tape and place it in the tray. 6. Insert the adhesive tape into the box. 7. Deposit the tape roll inside the receptacle. 8. Grab the tape and put it into the container.
Move Latte Cup (<i>PolaRiS</i>)	Put the latte art cup on top of the cutting board.	Relocate the pitcher with the decoration onto the wooden board.	1. Place the metal pitcher on the cutting board. 2. Move the latte cup onto the wooden board. 3. Set the silver cup on the cutting board. 4. Lift the milk jug and put it on the board. 5. Position the frothing pitcher on the wooden board. 6. Take the metal cup and place it on the cutting board. 7. Move the latte pitcher to the board. 8. Put the latte cup on top of the cutting board.
Carrot on Plate (<i>Real-World</i>)	Put carrot on plate.	Balance the carrot on the ceramic platter.	1. Place the carrot on the green dish. 2. Set the orange vegetable on the small plate. 3. Position the carrot upright on the platter. 4. Stand the carrot on the green plate. 5. Set the vegetable on the round dish. 6. Position the carrot on the small platter. 7. Balance the orange carrot on the green dish. 8. Place the vegetable on the ceramic plate.
Cube in Toolbox (<i>Real-World</i>)	Place the cube inside the toolbox.	Move the cube towards the container and put it inside.	1. Put the cube into the toolbox. 2. Insert the cube into the case. 3. Move the cube into the container. 4. Set the cube inside the red box. 5. Place the cube inside the red box. 6. Place the cube into the box. 7. Insert the cube in the open toolbox. 8. Put the cube inside the case.
Tape in Toolbox (<i>Real-World</i>)	Place the green tape inside the toolbox.	—	1. Put the green tape in the toolbox. 2. Set the tape inside the red box. 3. Move the green roll into the storage box. 4. Insert the green adhesive into the toolbox. 5. Place the colored tape in the red container. 6. Drop the green roll inside the toolbox. 7. Put the adhesive inside the storage box. 8. Set the tape in the container.
Screwdriver in Toolbox (<i>Real-World</i>)	Place the screwdriver tool inside the toolbox.	—	1. Put the screwdriver into the toolbox. 2. Insert the tool into the case. 3. Move the screwdriver into the container. 4. Set the tool inside the red box. 5. Place the tool inside the red box. 6. Place the green and black tool into the box. 7. Insert the screwdriver in the open toolbox. 8. Put the green-handled tool inside the case.